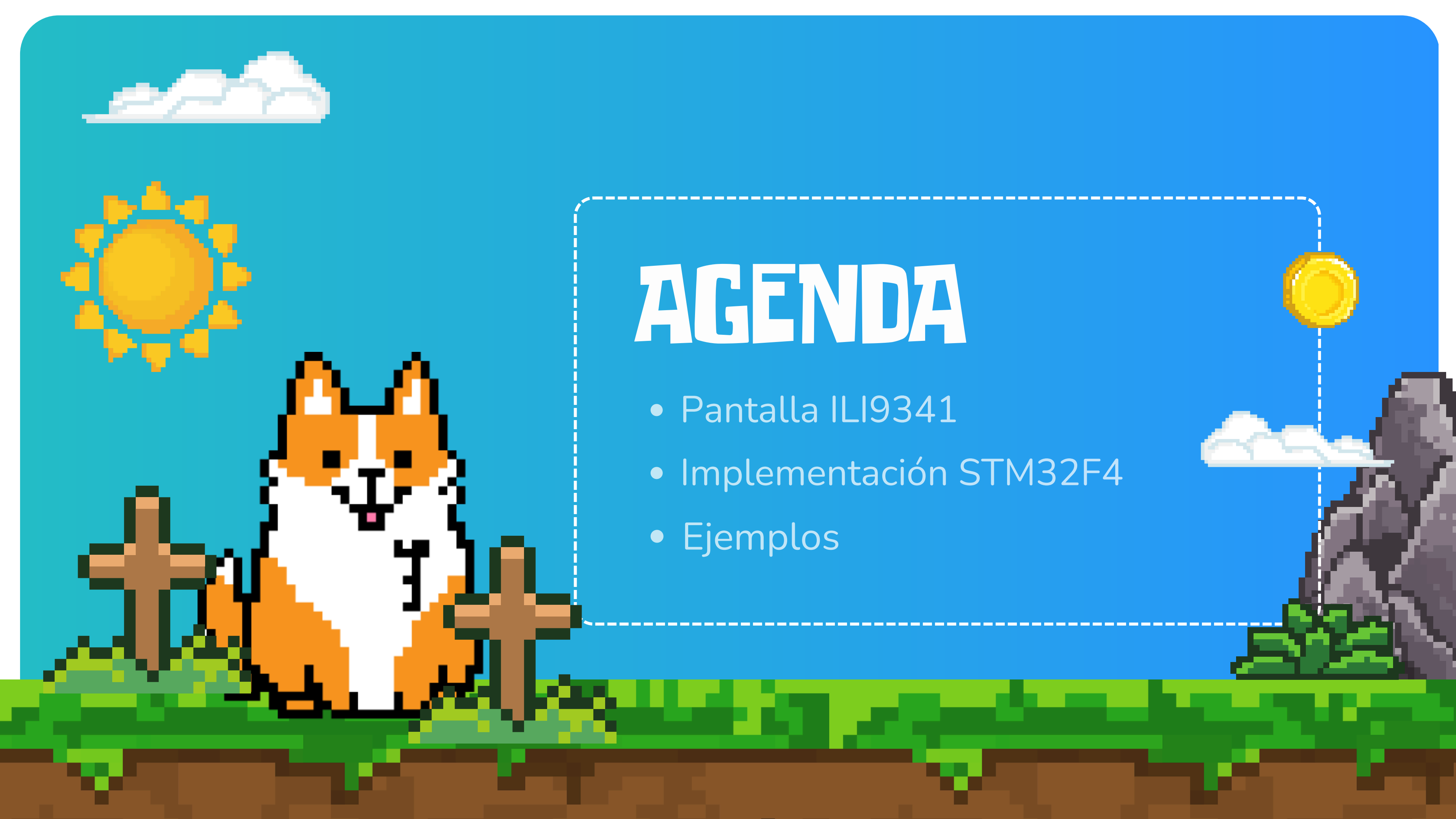


PANTALLA ili9341

Electrónica Digital 2





AGENDA

- Pantalla ILI9341
- Implementación STM32F4
- Ejemplos

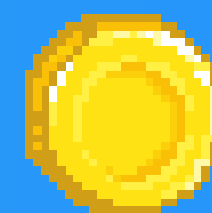
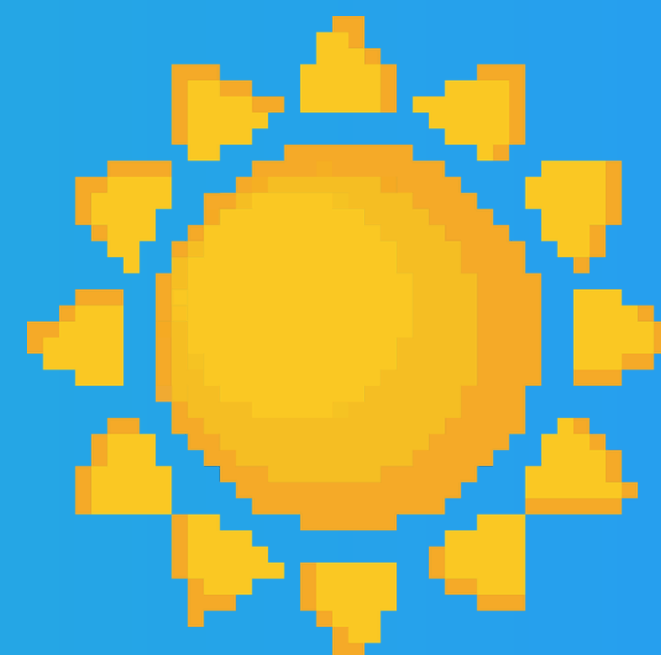


PANTALLA ili941

La pantalla LCD TFT posee un driver QVGA(Quarter VGA / cuarto de VGA) que es utilizado para controlar pantallas LCD VGA de 320x240.

Este driver lo encontramos normalmente en pantallas de bajo costo.

Tiene varios modos de interface SPI, 8, 9, 16, o 18 bits en interfaces paralelas.



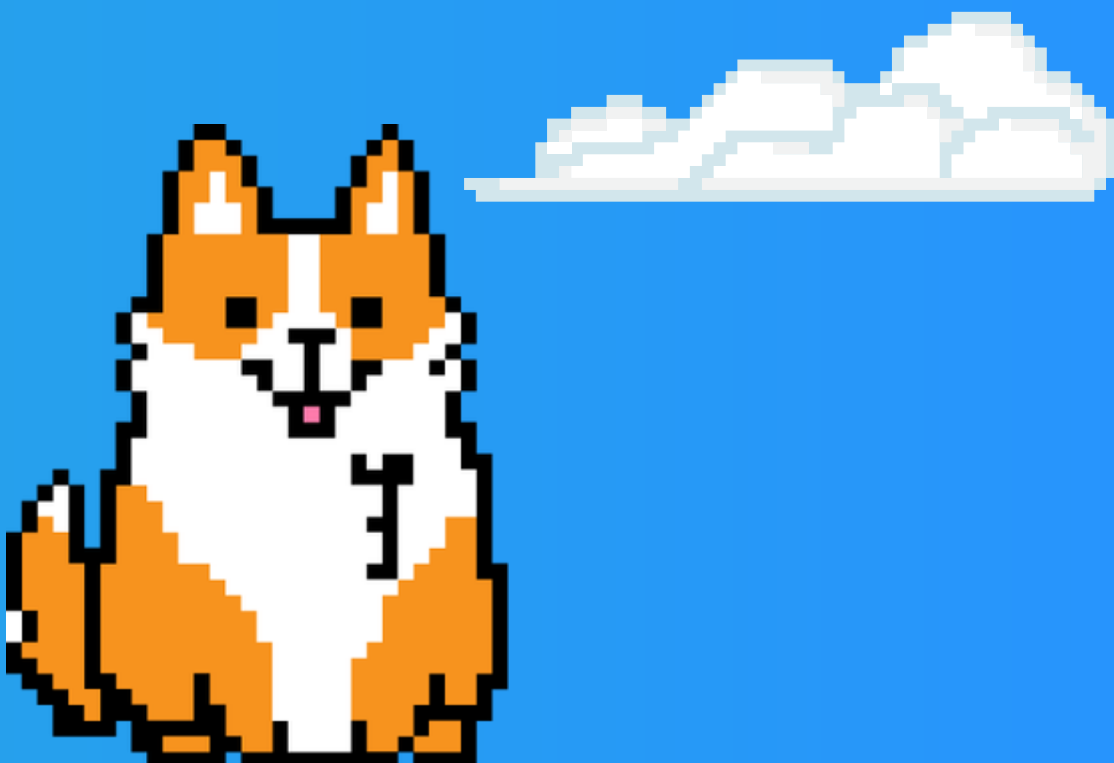


COLORES ili9341

Posee un arreglo de 320x240 pixeles = 76,800 pixeles

Tiene dos formatos de colores:

- 65K-Colors, RGB 5, 6, 5 -bits input data. (2 Bytes)
- 262K-Colors, RGB 6, 6, 6 -bits input data. (3 Bytes)



65K color: 16-bit/pixel (RGB 5-6-5 bits input)

One pixel (3 sub-pixels) display data is sent by 2 byte transfers when DBI [2:0] bits of 3Ah register are set to "101".

Count	0	1	2	3	4	...	477	478	479	480
D/CX	0	1	1	1	1	...	1	1	1	1
D7	C7	0R4	0G2	1R4	1G2	...	238R4	238G2	239R4	239G2
D6	C6	0R3	0G1	1R3	1G1	...	238R3	238G1	239R3	239G1
D5	C5	0R2	0G0	1R2	1G0	...	238R2	238G0	239R2	239G0
D4	C4	0R1	0B4	1R1	1B4	...	238R1	238B4	239R1	239B4
D3	C3	0R0	0B3	1R0	1B3	...	238R0	238B3	239R0	239B3
D2	C2	0G5	0B2	1G5	1B2	...	238G5	238B2	239G5	239B2
D1	C1	0G4	0B1	1G4	1B1	...	238G4	238B1	239G4	239B1
D0	C0	0G3	0B0	1G3	1B0	...	238G3	238B0	239G3	239B0

262K color: 18-bit/pixel (RGB 6-6-6 bits input)

One pixel (3 sub-pixels) display data is sent by 3 bytes transfer when DBI [2:0] bits of 3Ah register are set to "110".

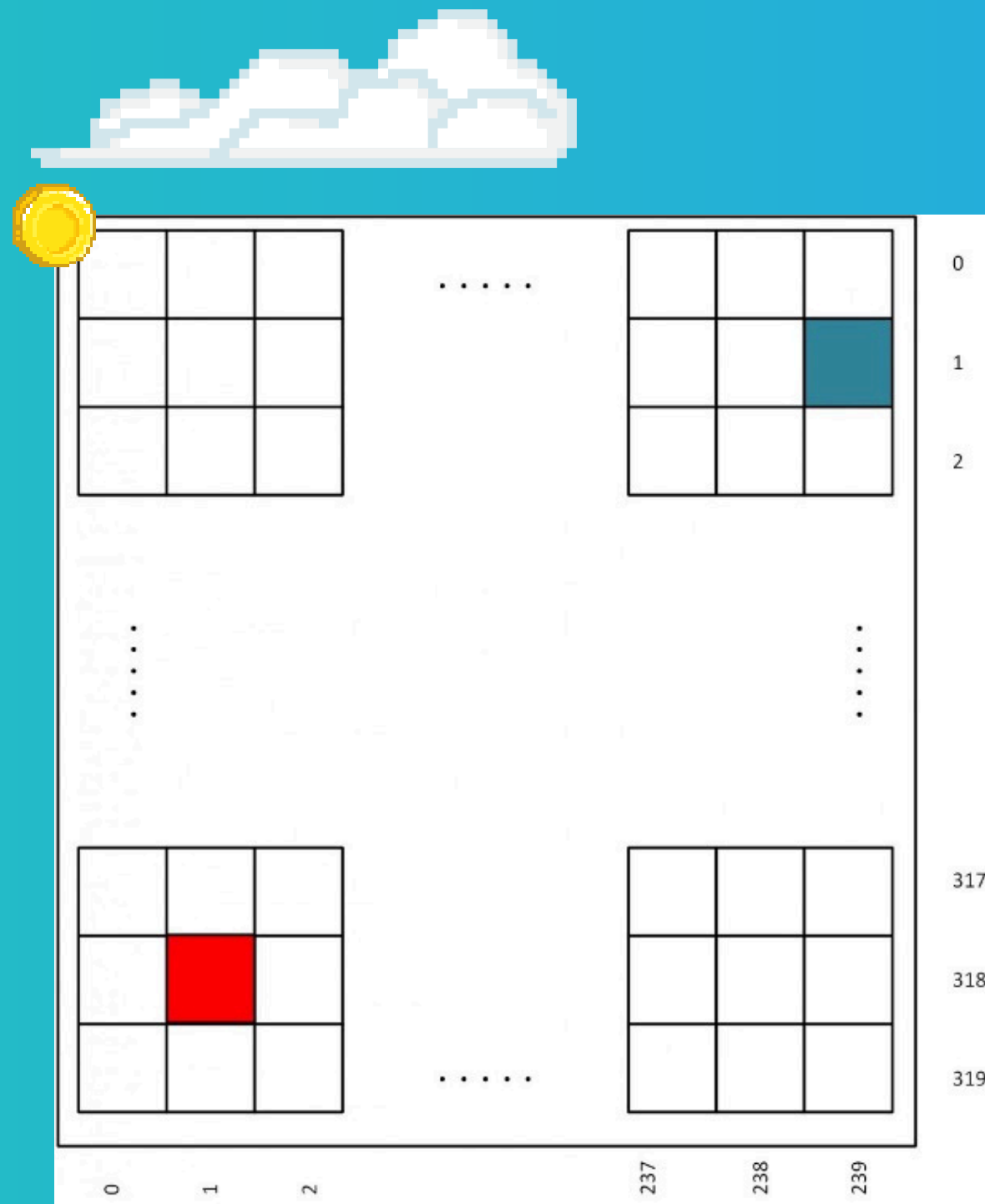
Count	0	1	2	3	...	718	719	720
D/CX	0	1	1	1	...	1	1	1
D7	C7	0R5	0G5	0B5	...	239R5	239G5	239B5
D6	C6	0R4	0G4	0B4	...	239R4	239G4	239B4
D5	C5	0R3	0G3	0B3	...	239R3	239G3	239B3
D4	C4	0R2	0G2	0B2	...	239R2	239G2	239B2
D3	C3	0R1	0G1	0B1	...	239R1	239G1	239B1
D2	C2	0R0	0G0	0B0	...	239R0	239G0	239B0
D1	C1				...			
D0	C0				...			

ORGANIZACIÓN

En modo vertical, la pantalla posee 240 pixels de ancho y 320 pixels de alto. Cada pixel es direccionado proporcionando las coordenadas X y Y (x,y).

El pixel 0,0 está ubicado en la esquina superior izquierda de la pantalla.

La orientación de la pantalla puede ser configurable mediante software





¡INTERFAZ DE 8 BITS (PARALELA)

La ventaja de trabajar la pantalla en este modo es que la velocidad de transmisión de imágenes es mucho más alta que la interface SPI.

La desventaja es que debemos disponer de 8 pines para los datos y 4 pines para señales de control.



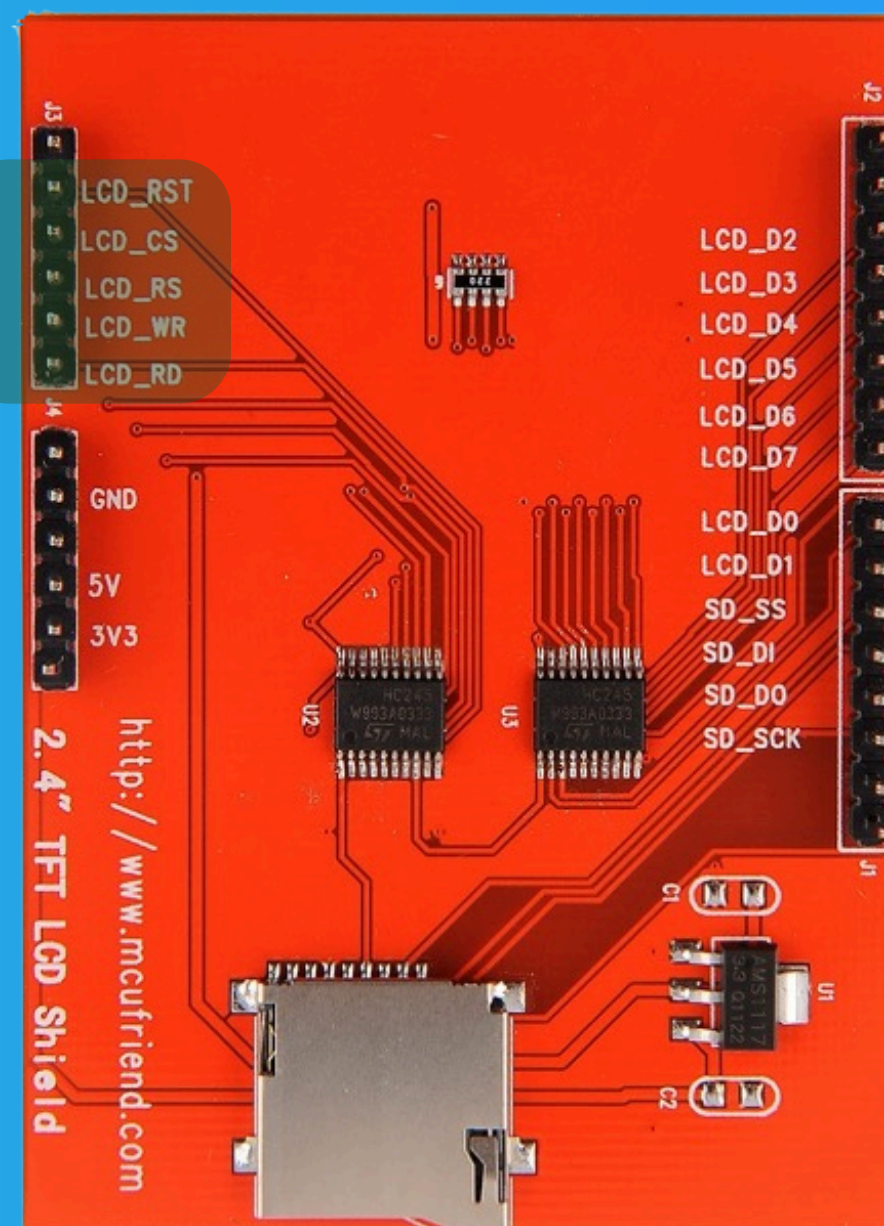


PiNES

CS: Es el pin de Chip Select, se activa en bajo (active low). El driver ILI9341 solo lee/escribe de la interfaz paralela cuando el CS esta en bajo.

WR: El pin de escritura (write) se activa en bajo. Cuando /WR esta en 0 lógico, el valor en la interfaz es escrita al driver.

Control



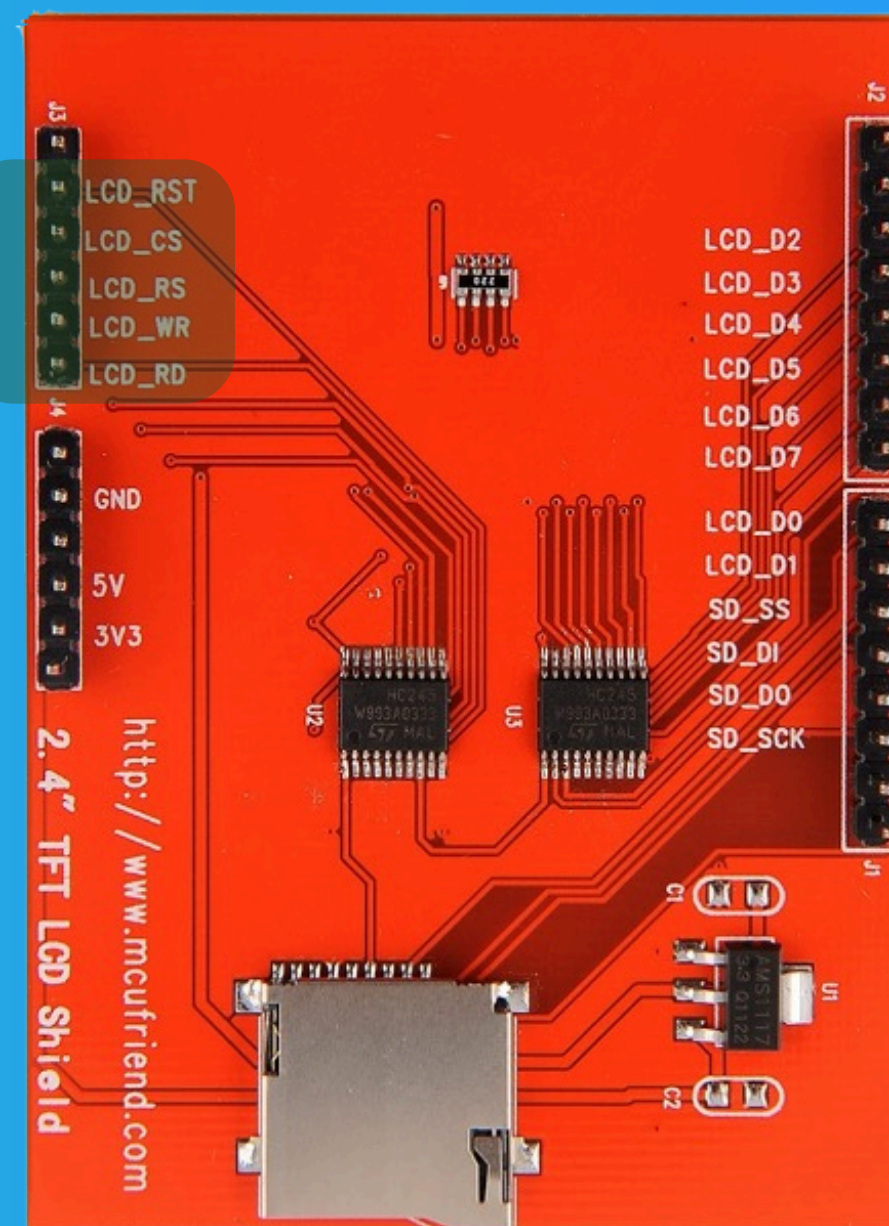


PiNES

RD: El pin de lectura (read) se activa en bajo. Cuando /RD esta en 0 lógico, el driver coloca el contenido en la interfaz.

RS: El pin de datos/comandos nos permite especificar si el dato colocado en la interfaz es comando o un dato (0 = comando / 1 = dato).

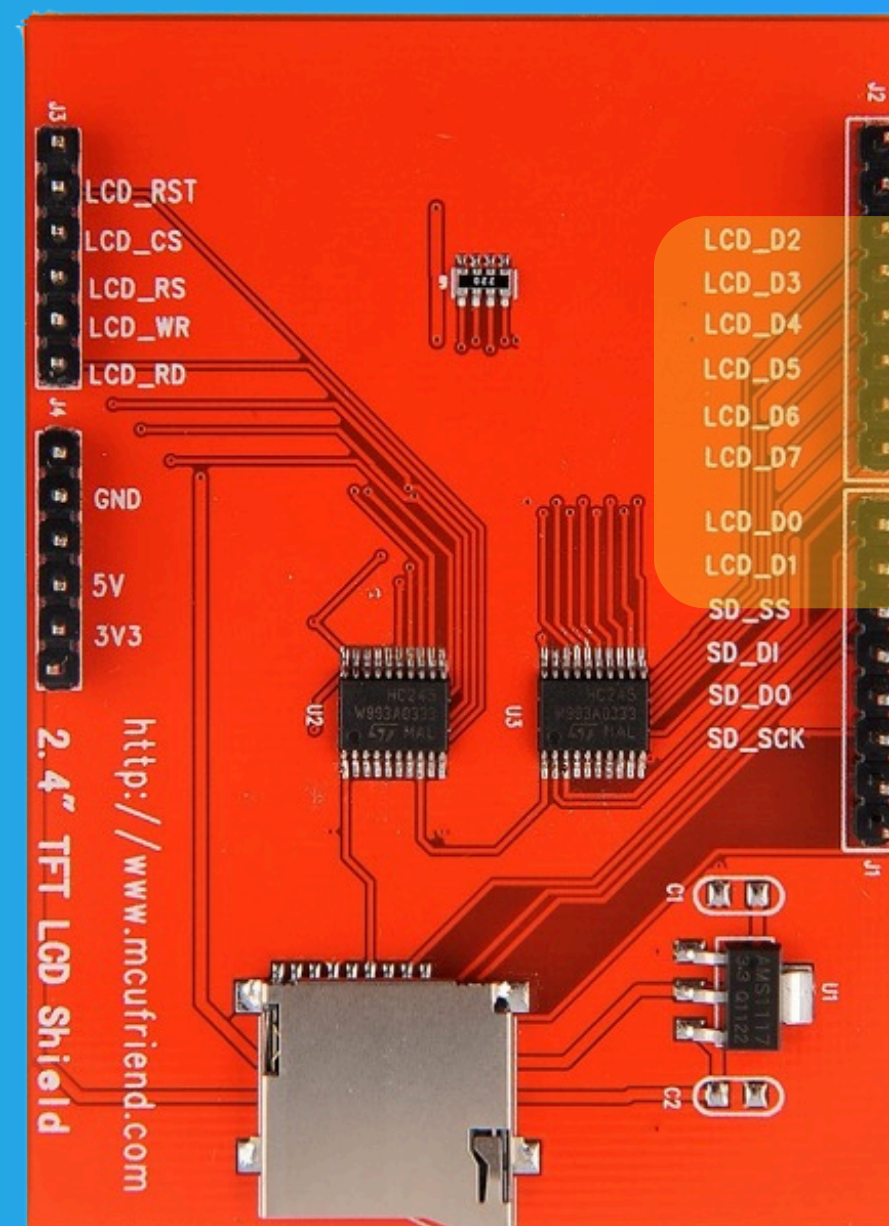
Control





PiNES

D[7:0] La interfaz de datos es una interfaz paralela de 8 bits que se utiliza para suministrar los valores que se escribirán al driver ILI9341.



Datos

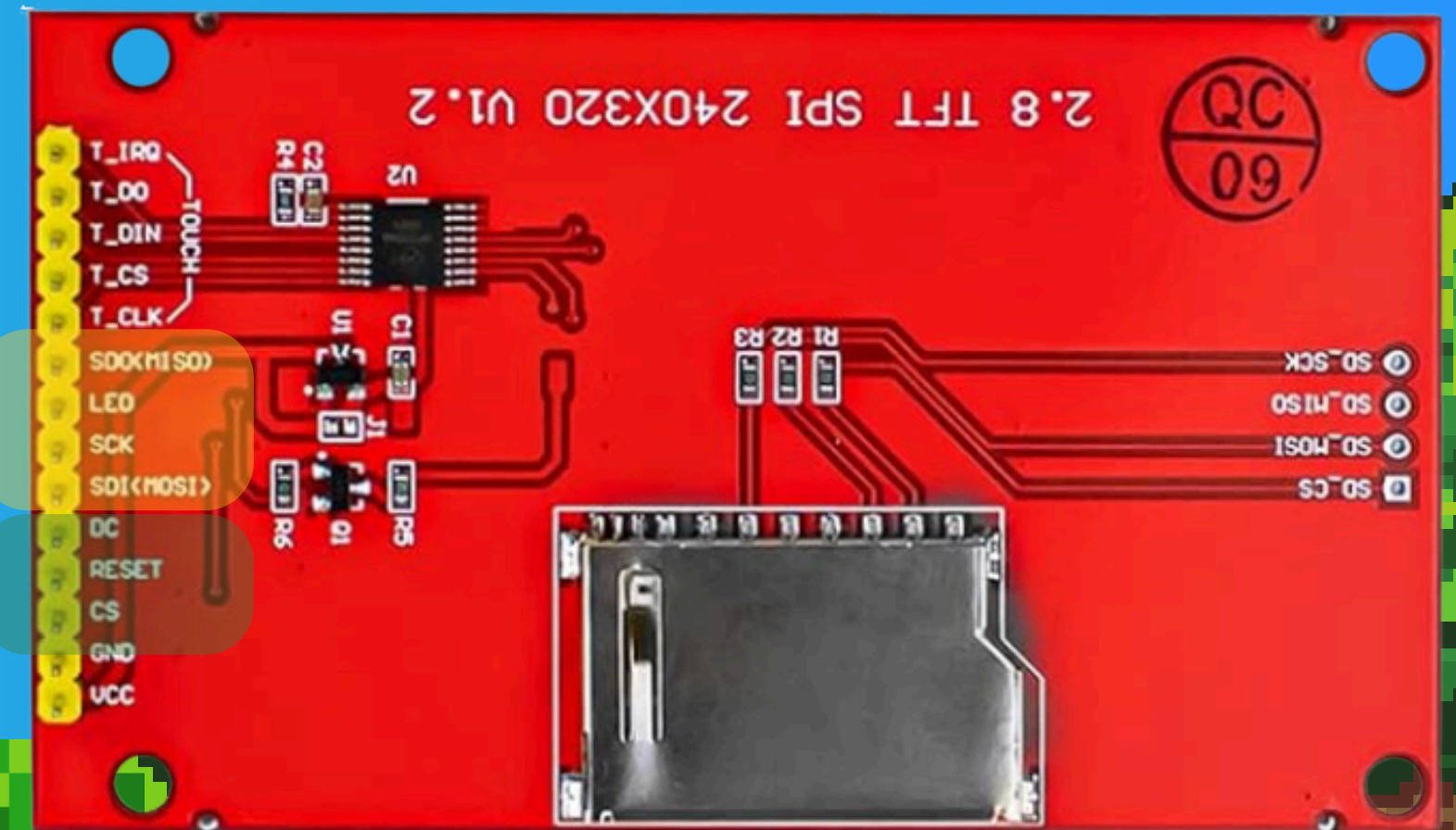


¡INTERFAZ SPI

La ventaja de trabajar la pantalla en este modo es que no necesitamos tantos pines para implementarla, 4 del SPI y 3 mas de control.



SPI
Control





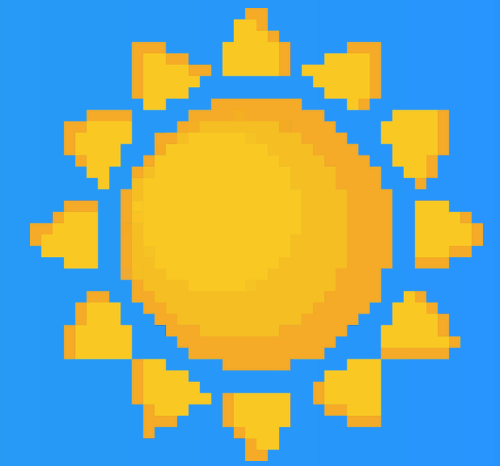
SET DE COMANDOS

Posee una gran lista de comandos para poder manejar el comportamiento de la pantalla.

Command Function	D/CX	RDX	WRX	D17-8	D7	D6	D5	D4	D3	D2	D1	D0	Hex
Display OFF	0	1	↑	XX	0	0	1	0	1	0	0	0	28h
Display ON	0	1	↑	XX	0	0	1	0	1	0	0	1	29h
Column Address Set	0	1	↑	XX	0	0	1	0	1	0	1	0	2Ah
	1	1	↑	XX	SC [15:8]								XX
	1	1	↑	XX	SC [7:0]								XX
	1	1	↑	XX	EC [15:8]								XX
	1	1	↑	XX	EC [7:0]								XX
Page Address Set	0	1	↑	XX	0	0	1	0	1	0	1	1	2Bh
	1	1	↑	XX	SP [15:8]								XX
	1	1	↑	XX	SP [7:0]								XX
	1	1	↑	XX	EP [15:8]								XX
	1	1	↑	XX	EP [7:0]								XX



ESCRIBIENDO COMANDOS



1.

CSX
(Chip Select)

2.

D/CX
(Data/Command)

4.

WRX
(Data/Command)

3.

D[7:0]

0x2A
(Column Address Set)

Pixelion



ESCRIBIENDO DATOS

1.

CSX
(Chip Select)

2.

D/CX
(Data/Command)

4.

WRX
(Data/Command)

5.

D[7:0]

DATA



ESCRIBIENDO DATOS

1.

CSX
(Chip Select)



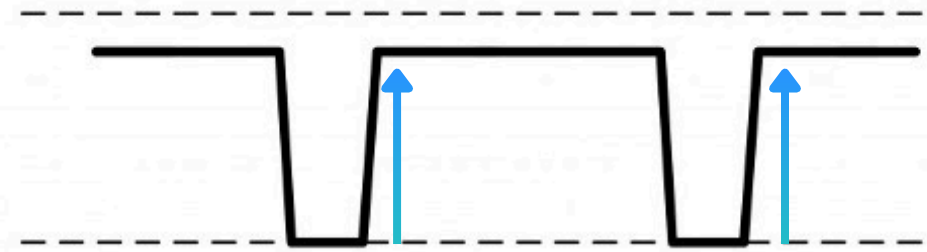
2.

D/CX
(Data/Command)



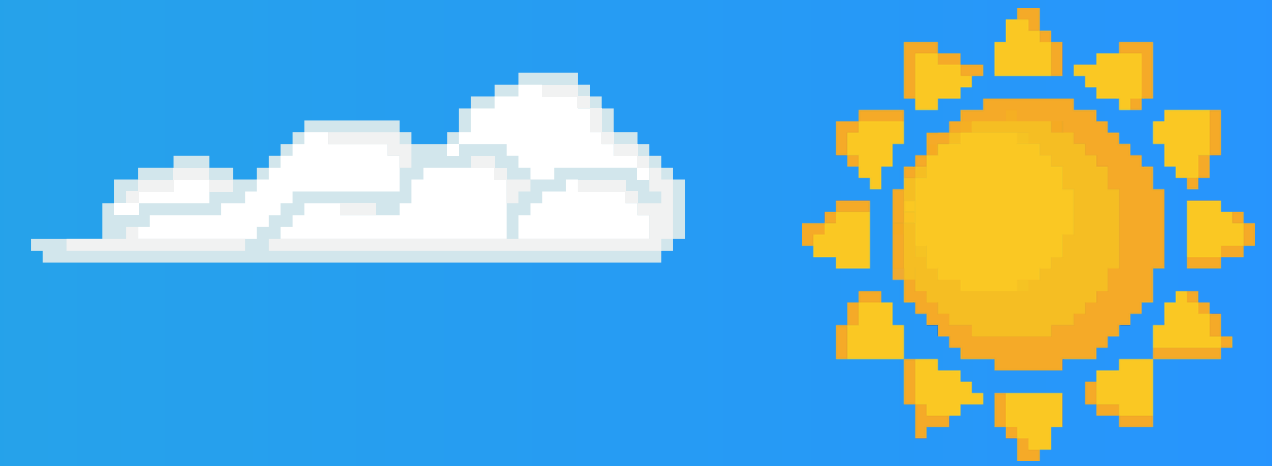
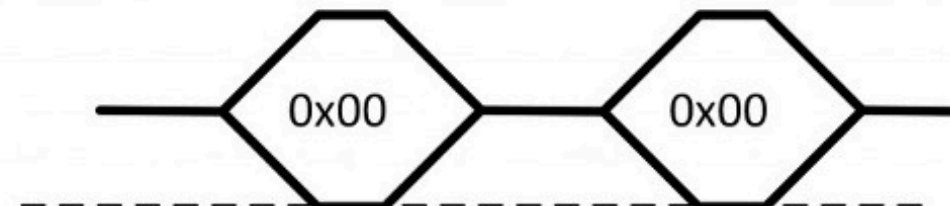
4.

WRX
(Data/Command)



3.

D[7:0]



PARA DIBUJAR

Es necesario definir el área de trabajo (ventana)

Paso 1. Definir Dirección de columnas (x)

Paso 2. Definir Dirección de páginas (y)

Columnas: $x_i \rightarrow x_f$

Páginas: $y_i \rightarrow y_f$

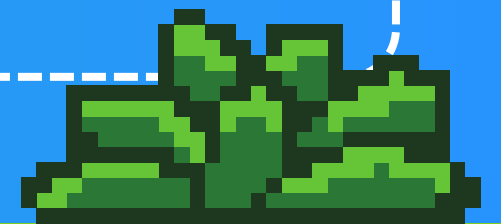
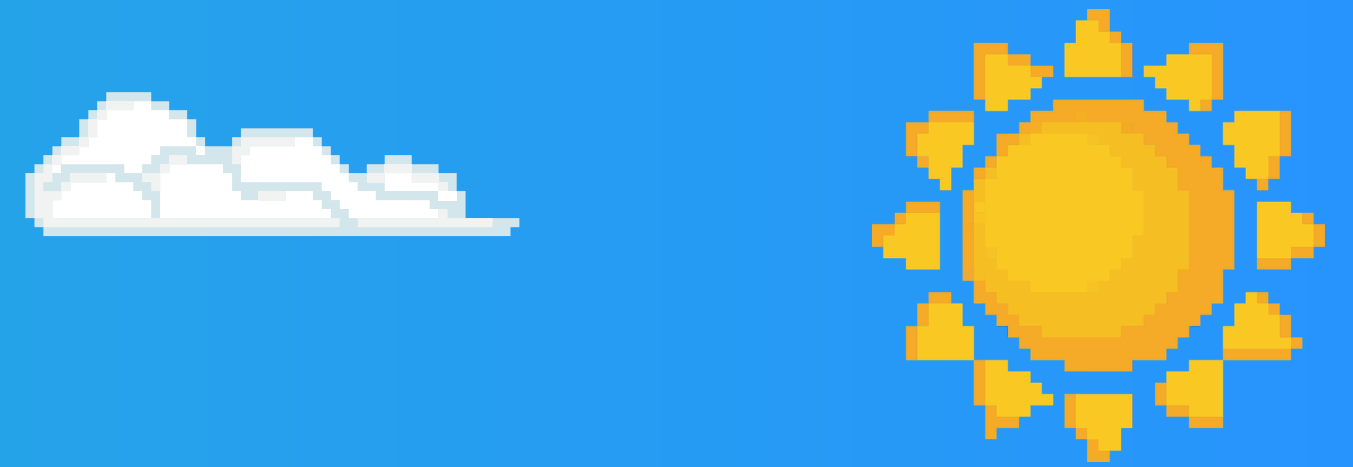
0,0

240

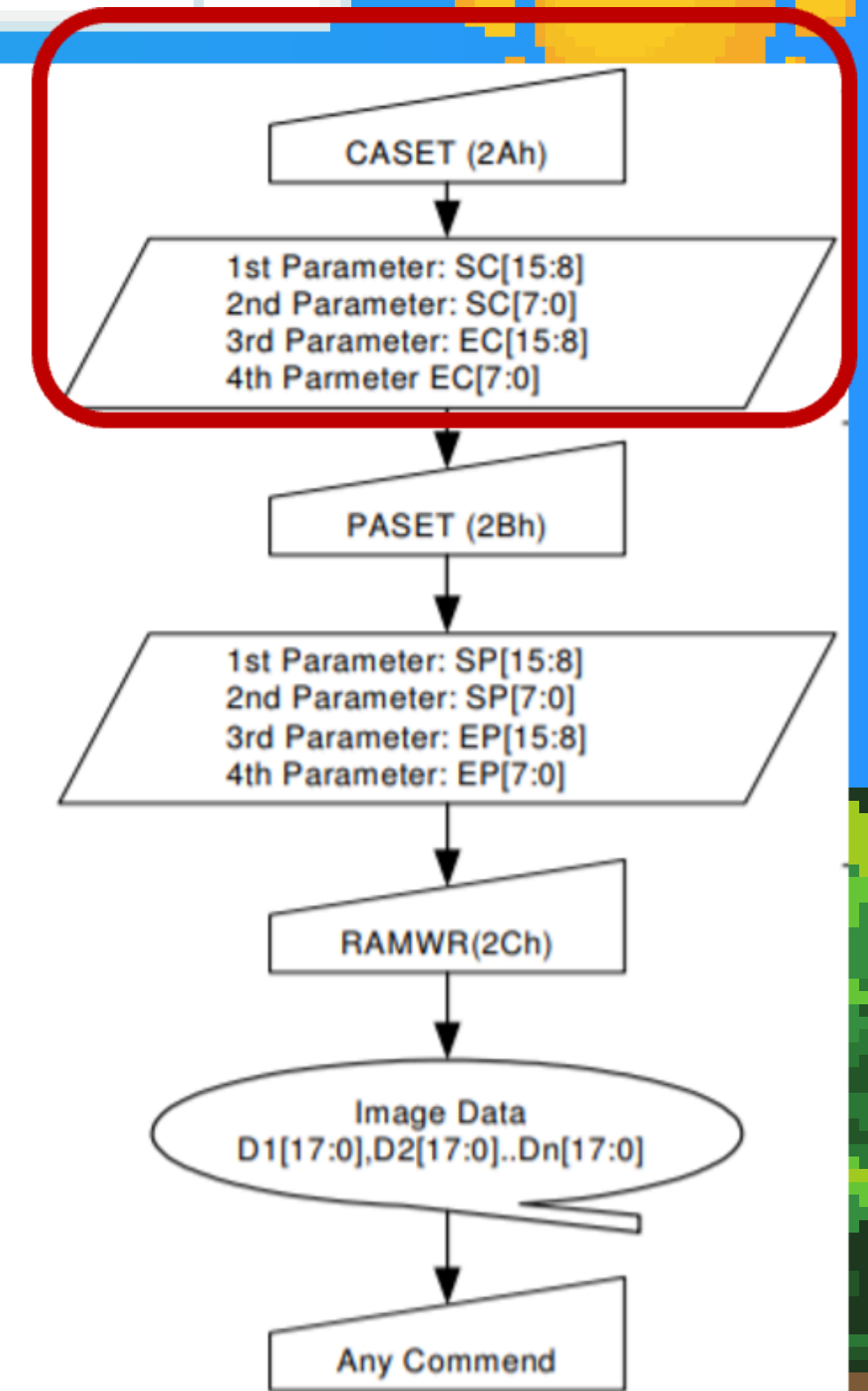
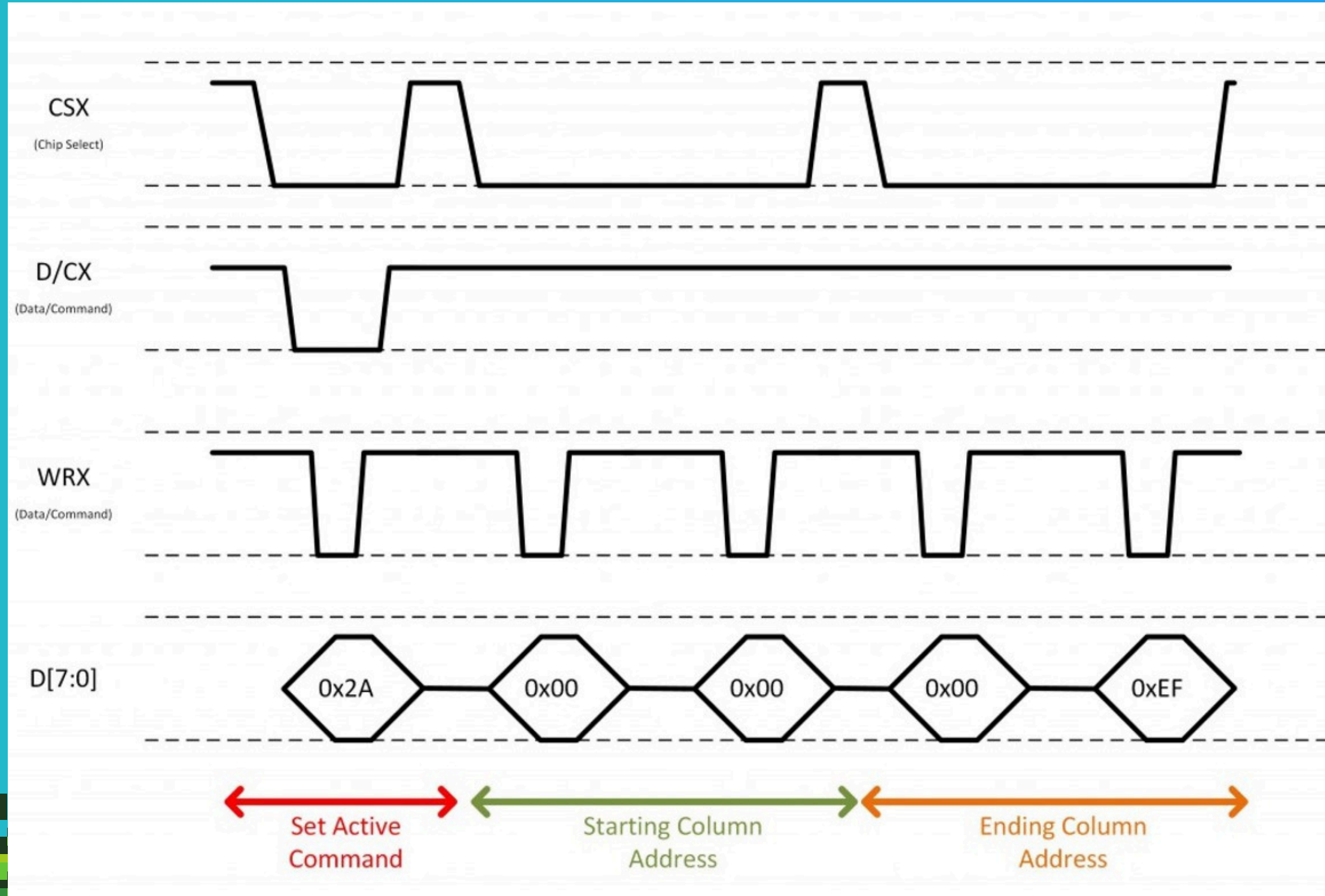
320

x_i, y_i

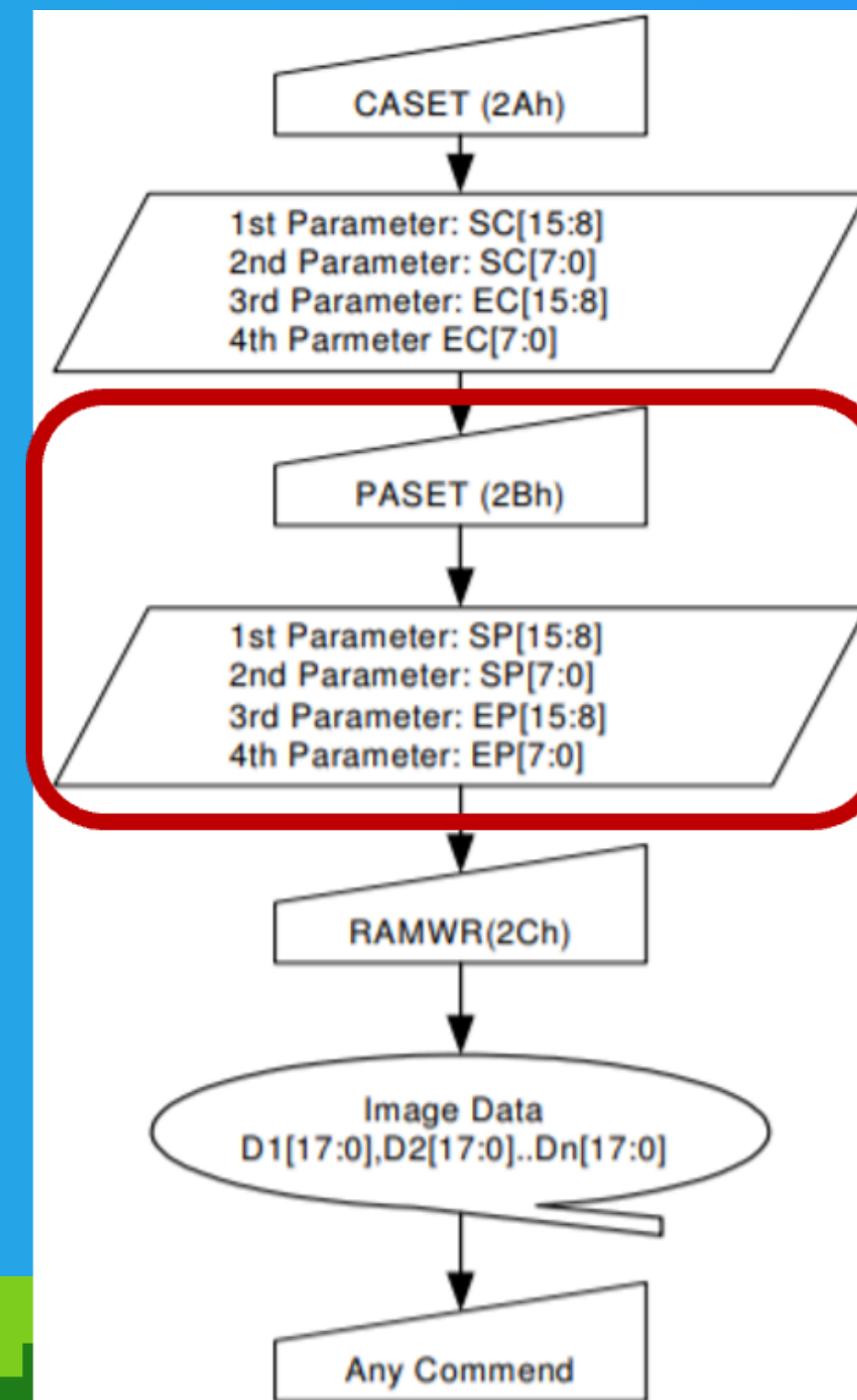
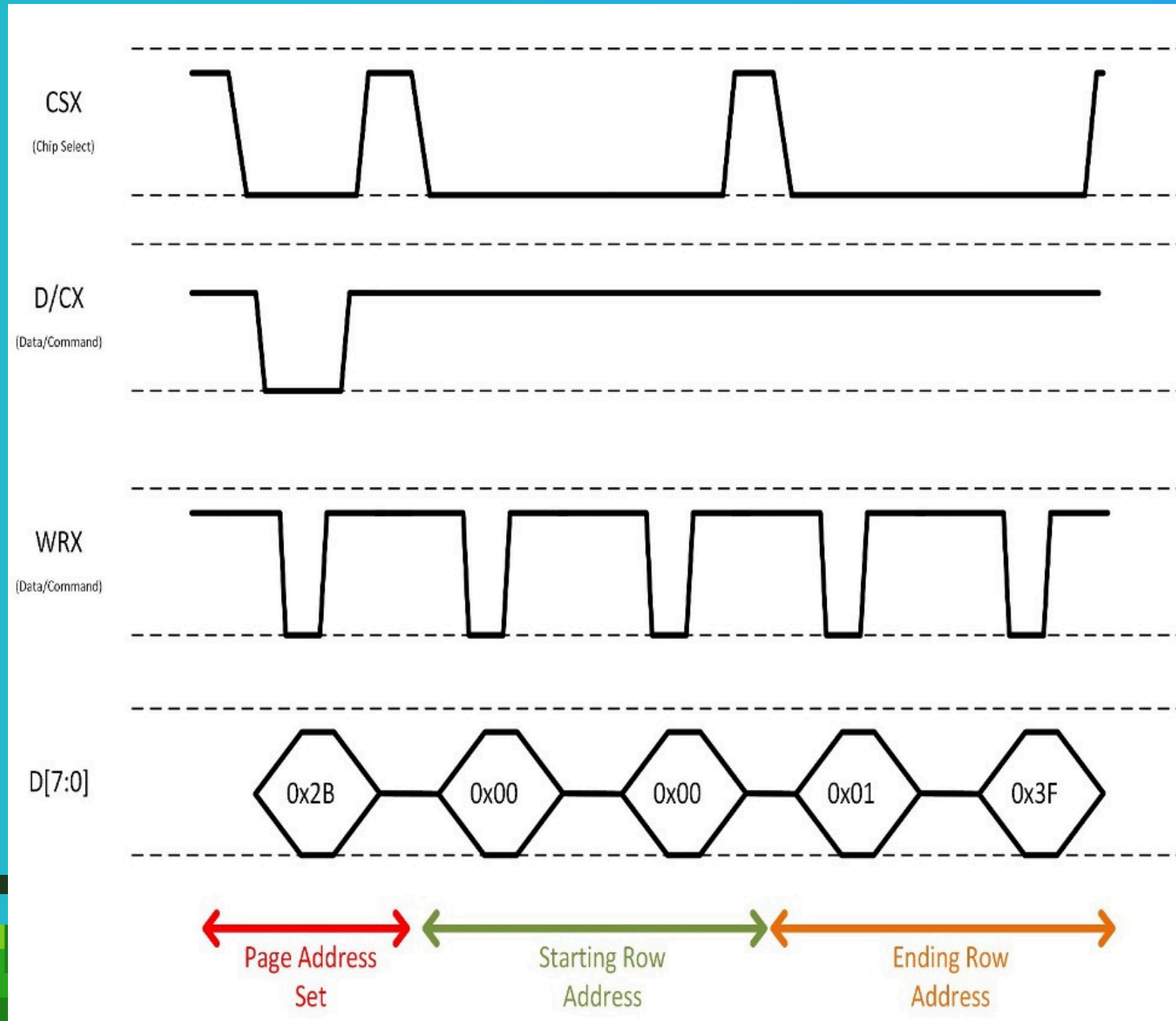
x_f, y_f



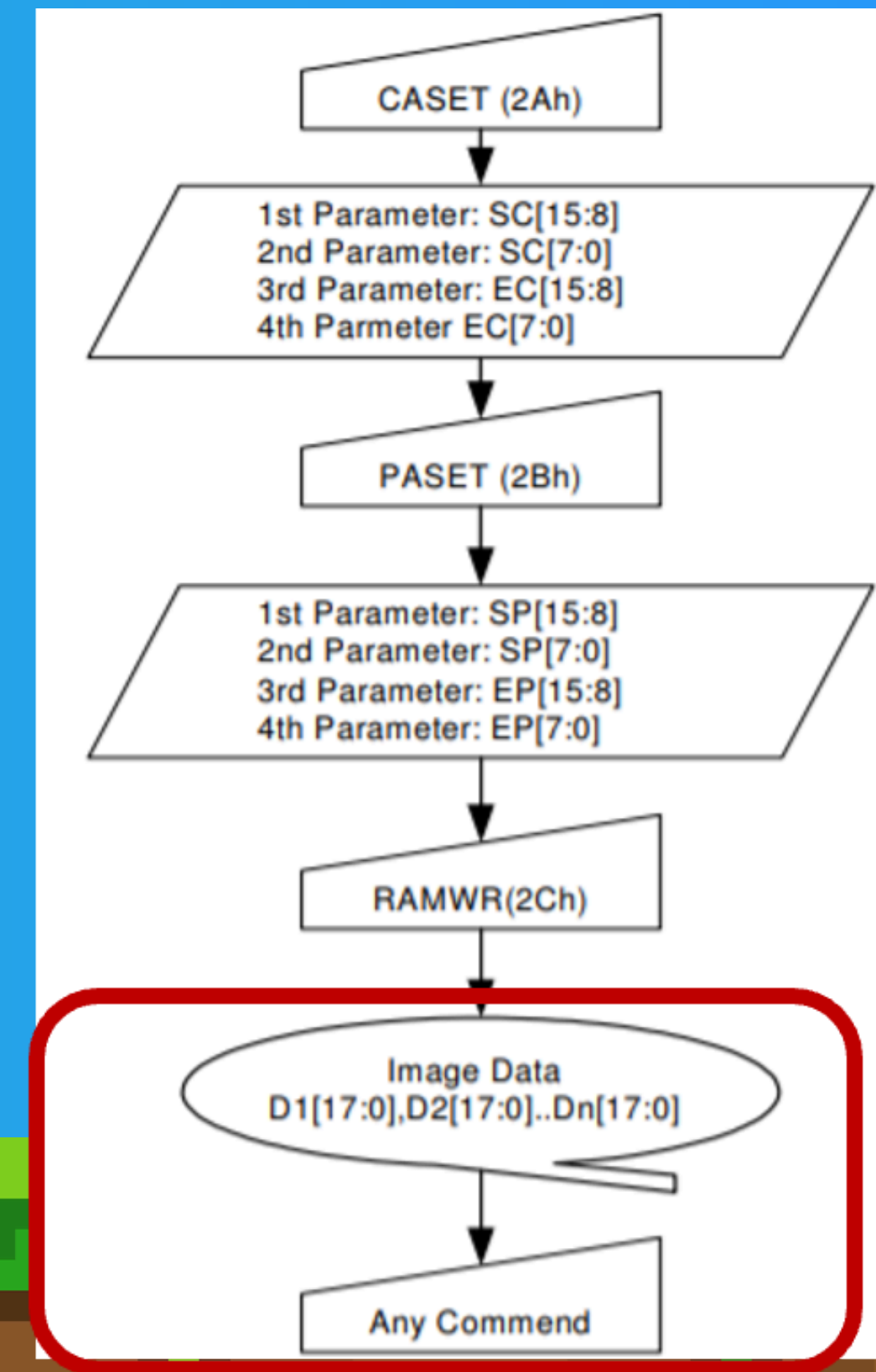
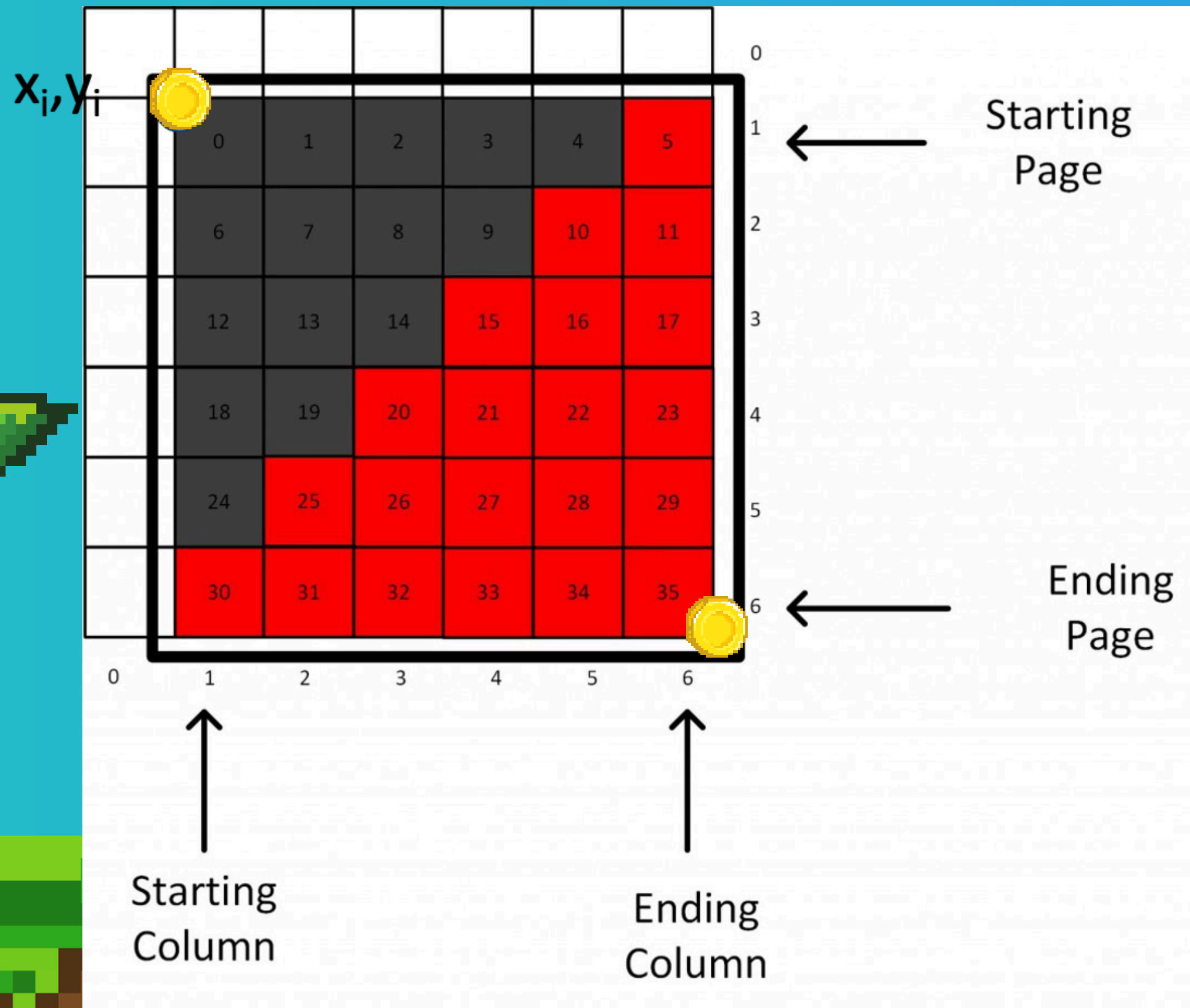
DEFINICIÓN DE COLUMNAS



DEFINICIÓN DE PÁGINAS



PINTANDO EL DIBUJO



¿CÓMO SE GENERAN LAS IMÁGENES?

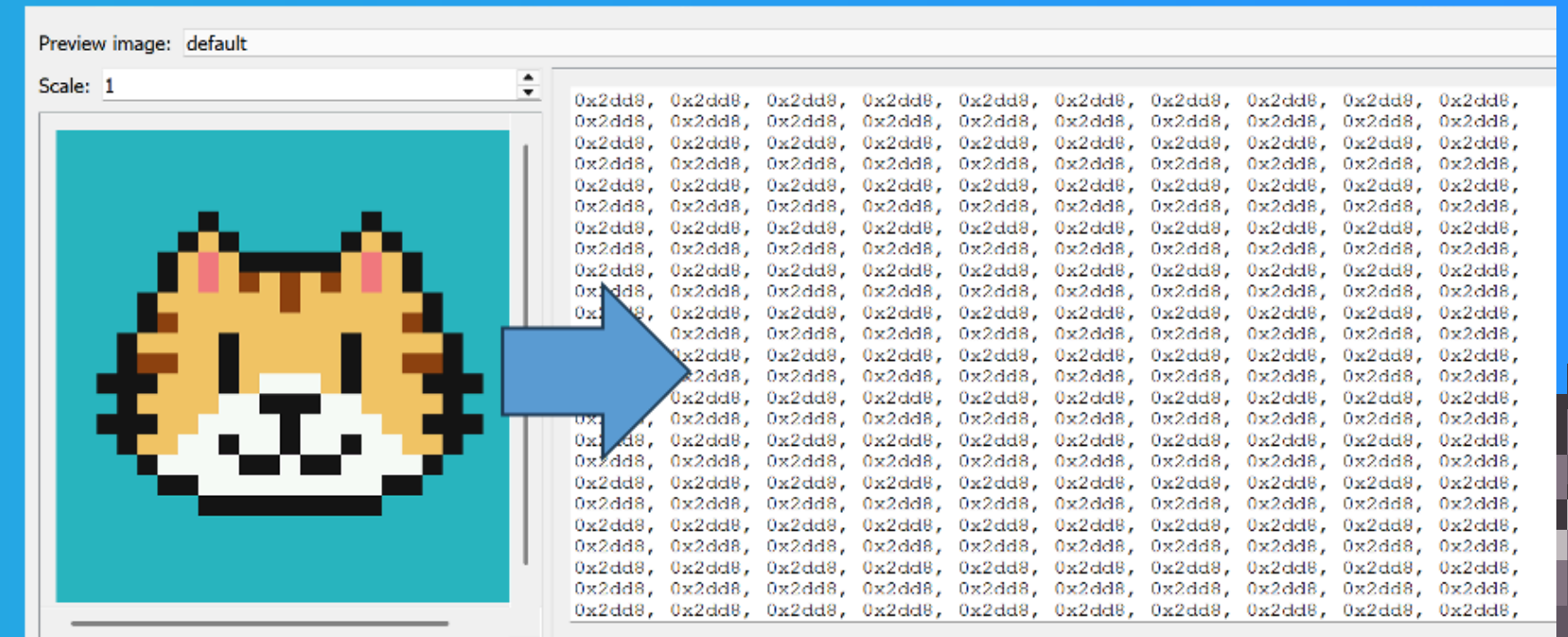
La pantalla no entiende imágenes como archivo, sino como una secuencia de datos de colores.

La pantalla NO usa formatos como JPG o PNG

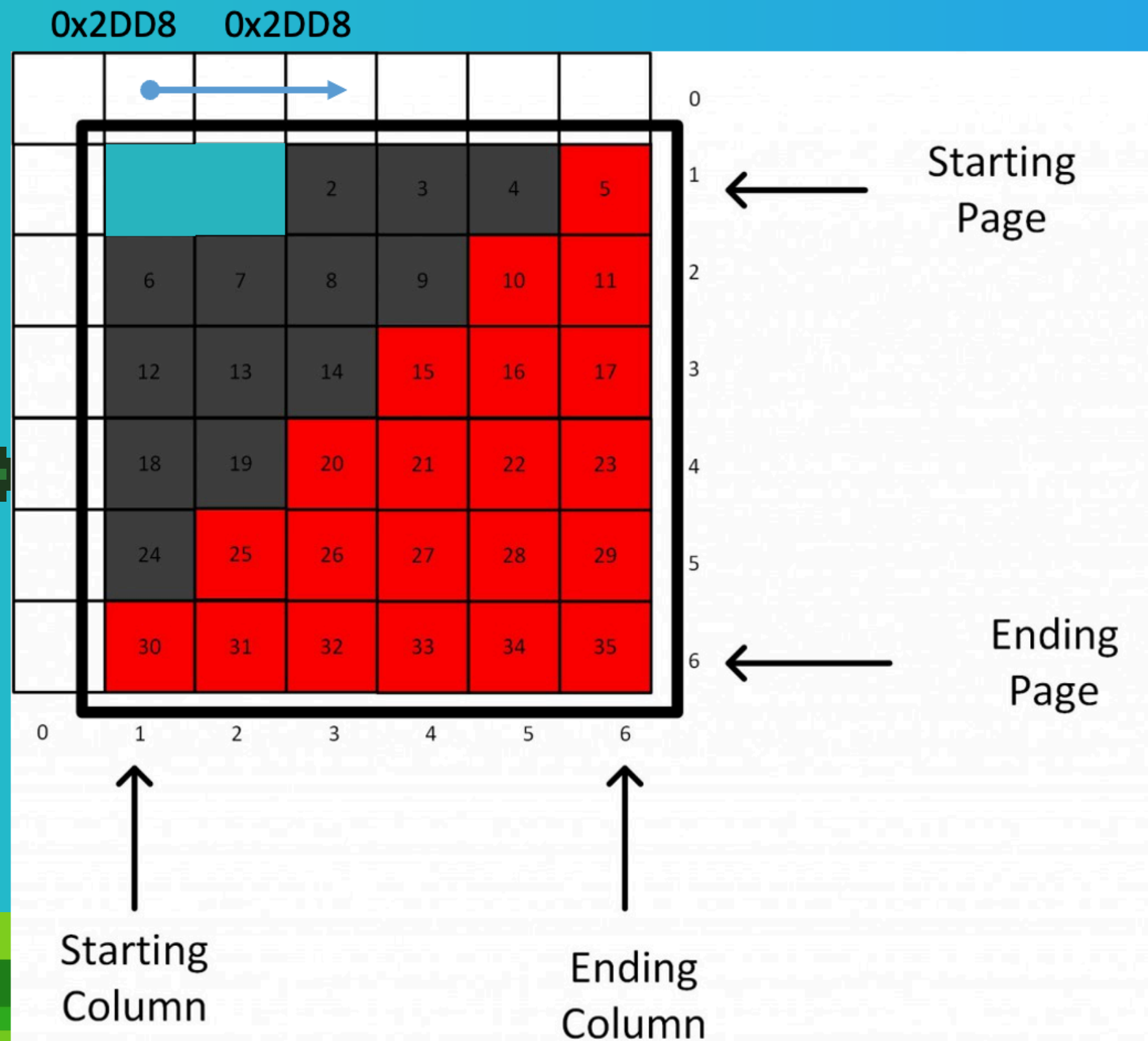
Se requiere convertir la imagen a datos en C

Cada pixel es un valor en formato RGB565 (2 bytes)

La imagen se representa como un arreglo



PINTANDO EL DIBUJO

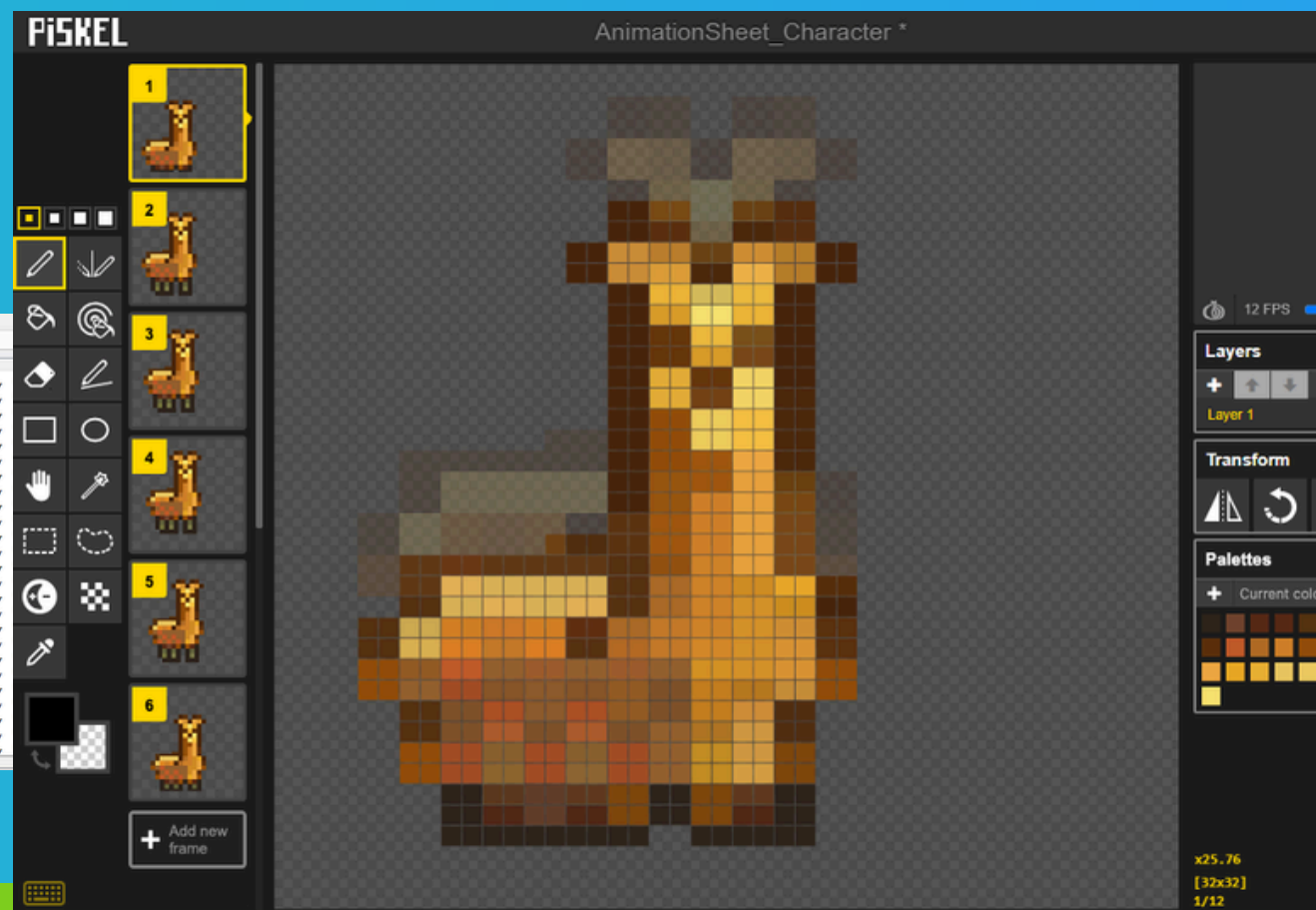
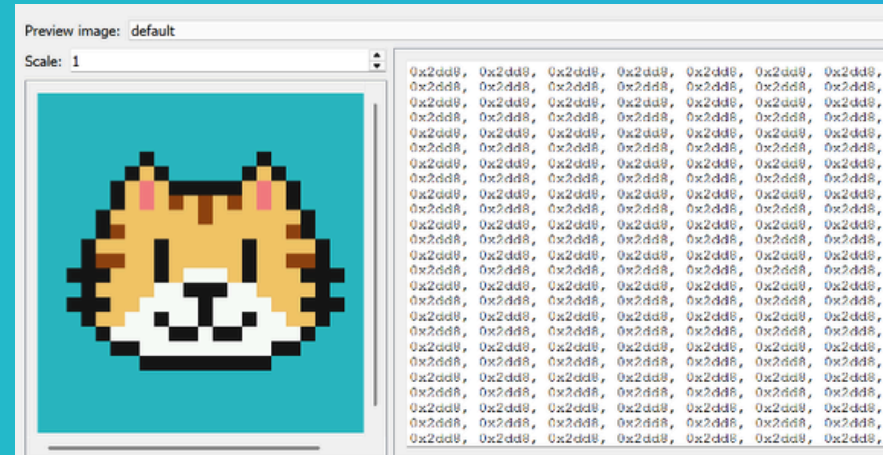


Empezamos a mandar 2 bytes que representan el color que deseamos pintar

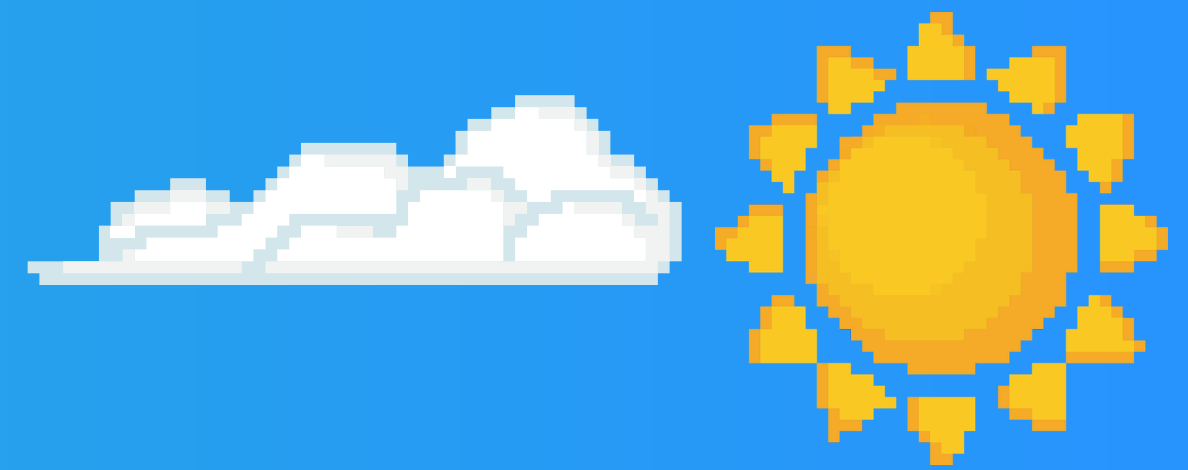


HERRAMIENTAS PARA TRABAJAR CON IMÁGENES

- Piskel (para pixel art)
- Photoshop / GIMP (opcional)
- LCD Image Converter



FLUJO DE MANEJO DE IMÁGENES



Para editar podemos usar un editor de imágenes como Piskelapp

Con LCD Image Converter generamos directamente el arreglo listo para usar en el microcontrolador.



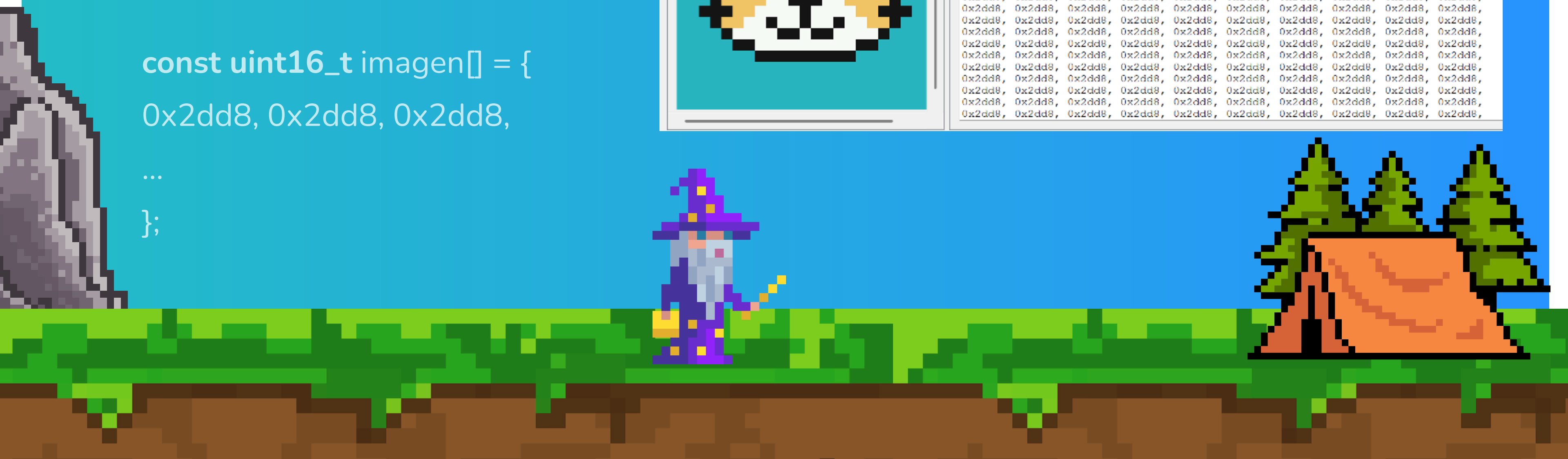
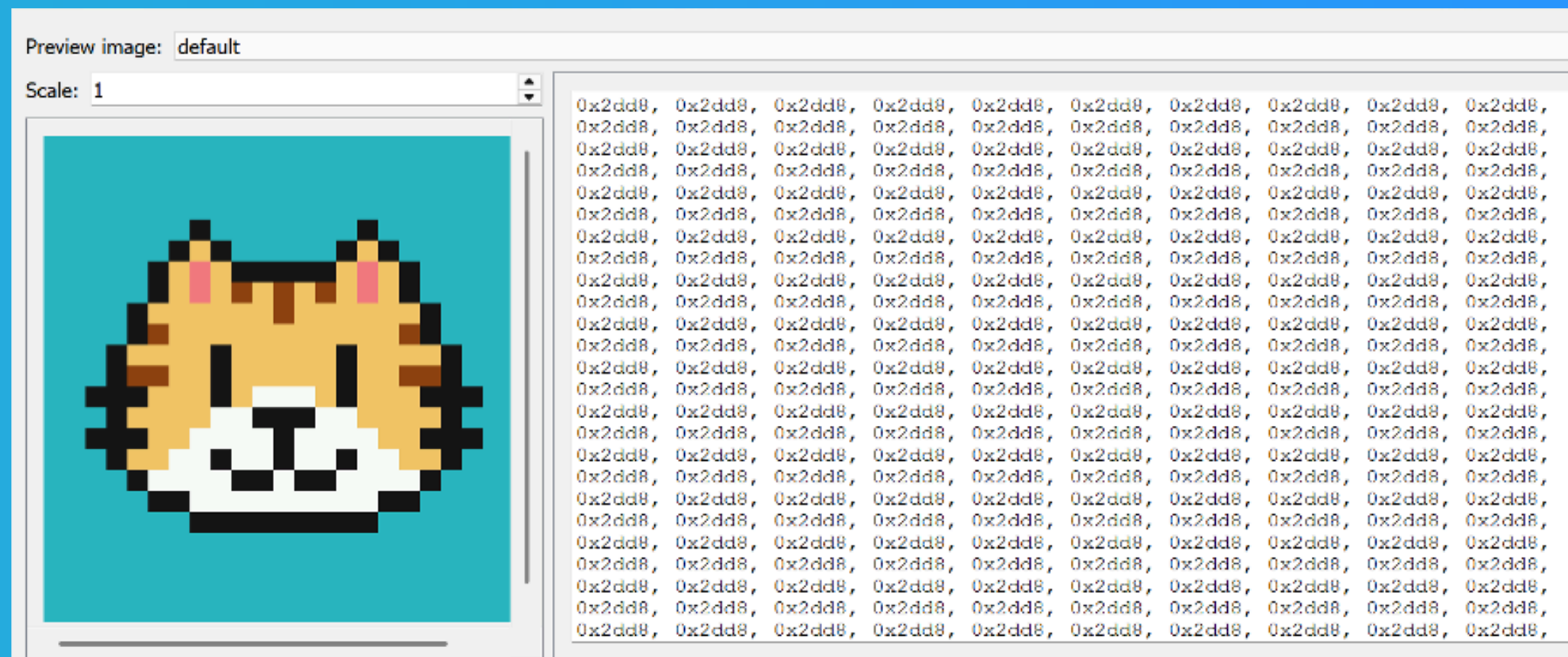
¿CÓMO SE VE UNA IMAGEN EN CÓDIGO?

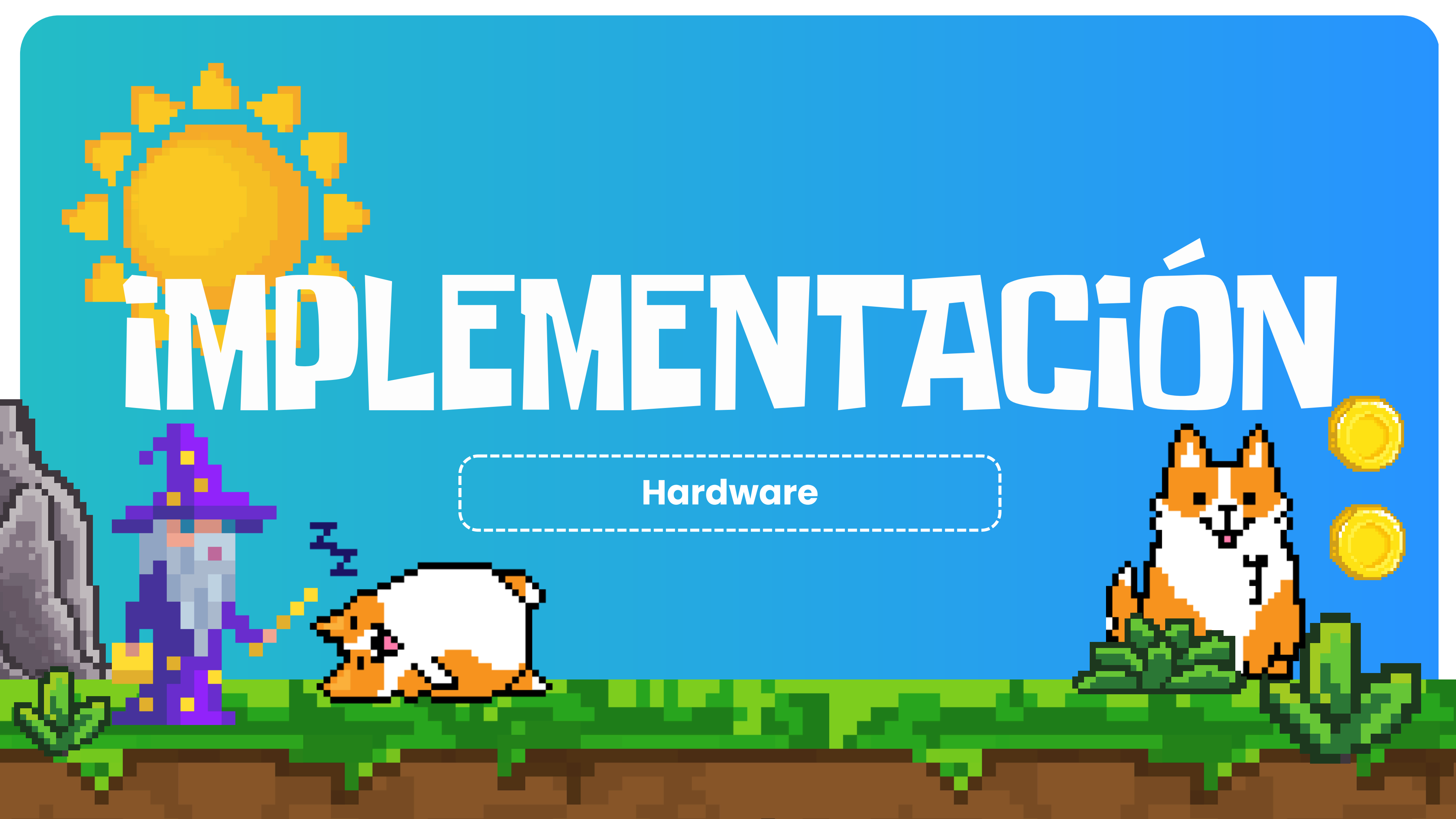
Cada valor = un pixel

Formato RGB565

Se envía a la pantalla secuencialmente

```
const uint16_t imagen[] = {
    0x2dd8, 0x2dd8, 0x2dd8,
    ...
};
```



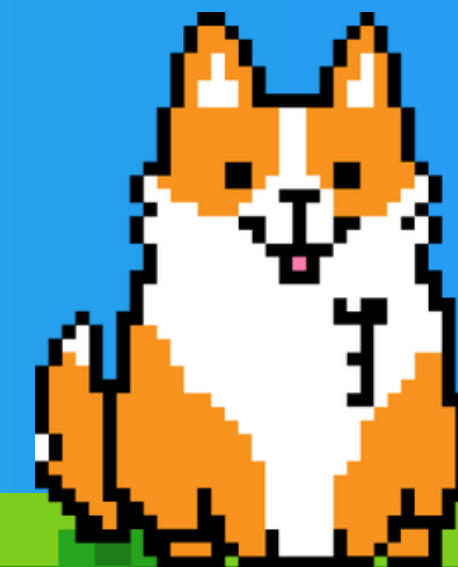
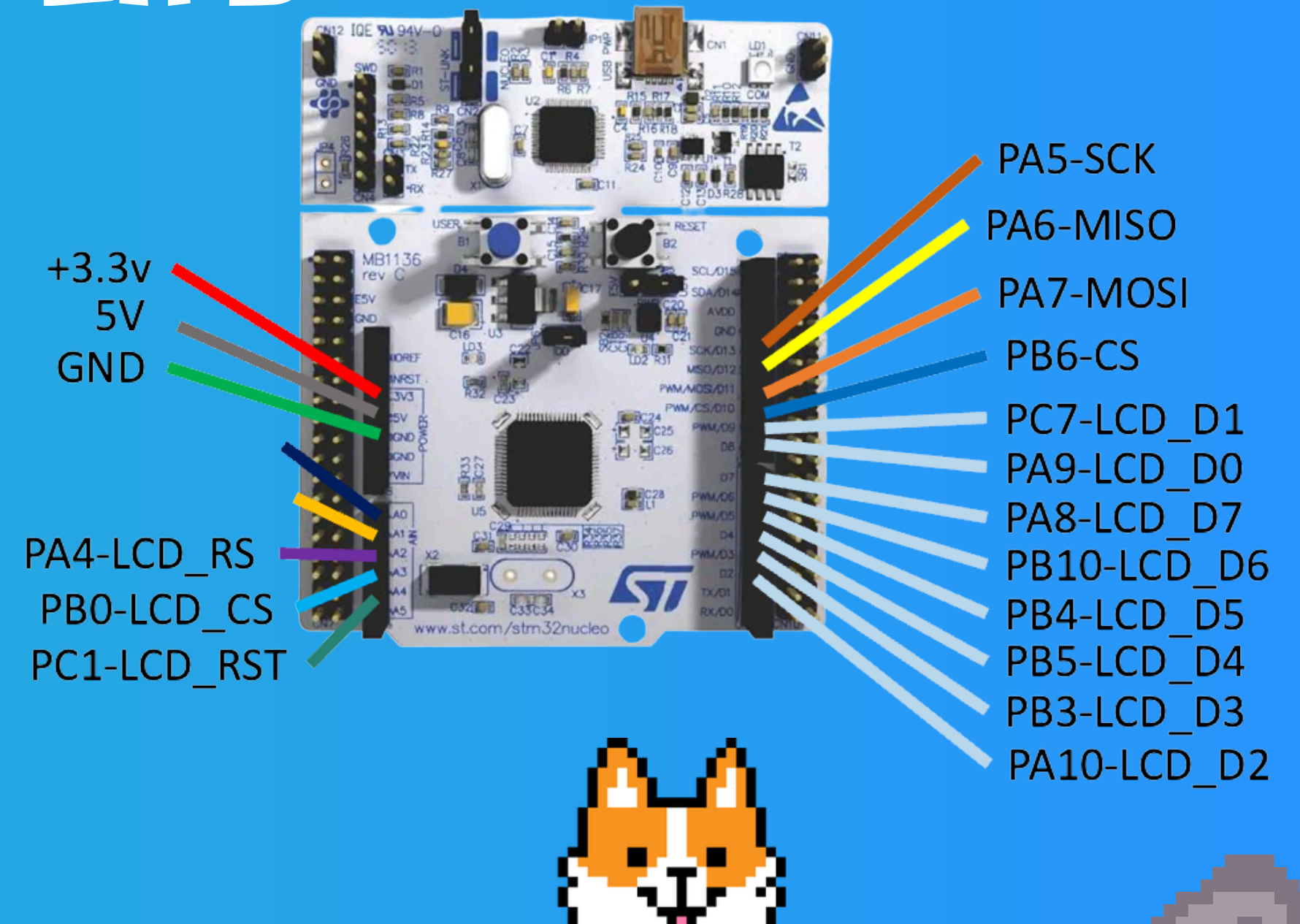
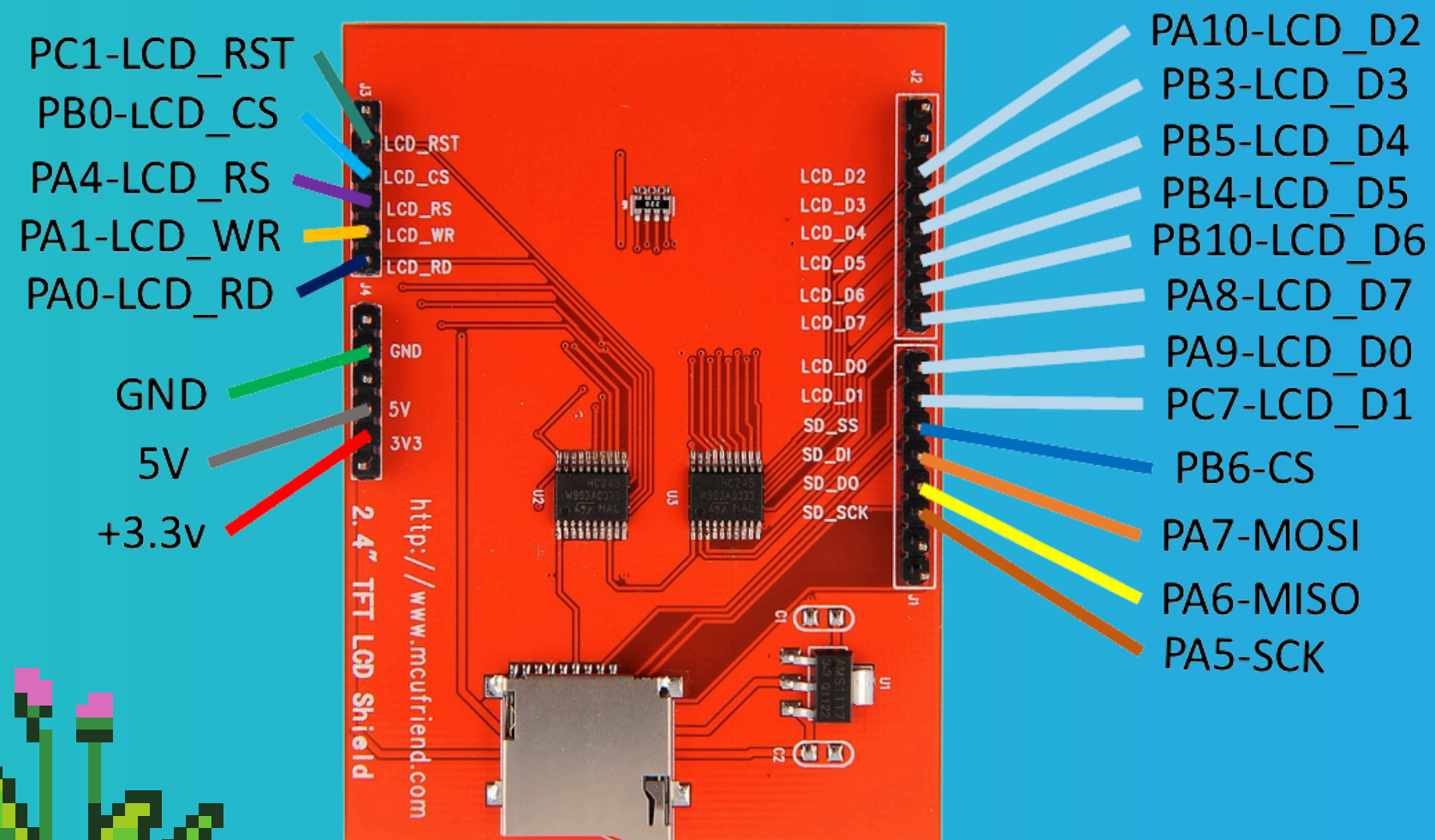
The background is a pixel art scene. At the top left is a large yellow sun with rays. Below it on the left is a wizard in a purple robe and hat, holding a staff. In the center is a white dog with orange ears and tail, lying down. On the right is an orange and white cat sitting on a green bush. Two gold coins are floating above the cat. The ground is green grass with brown soil at the bottom.

IMPLEMENTACIÓN

Hardware

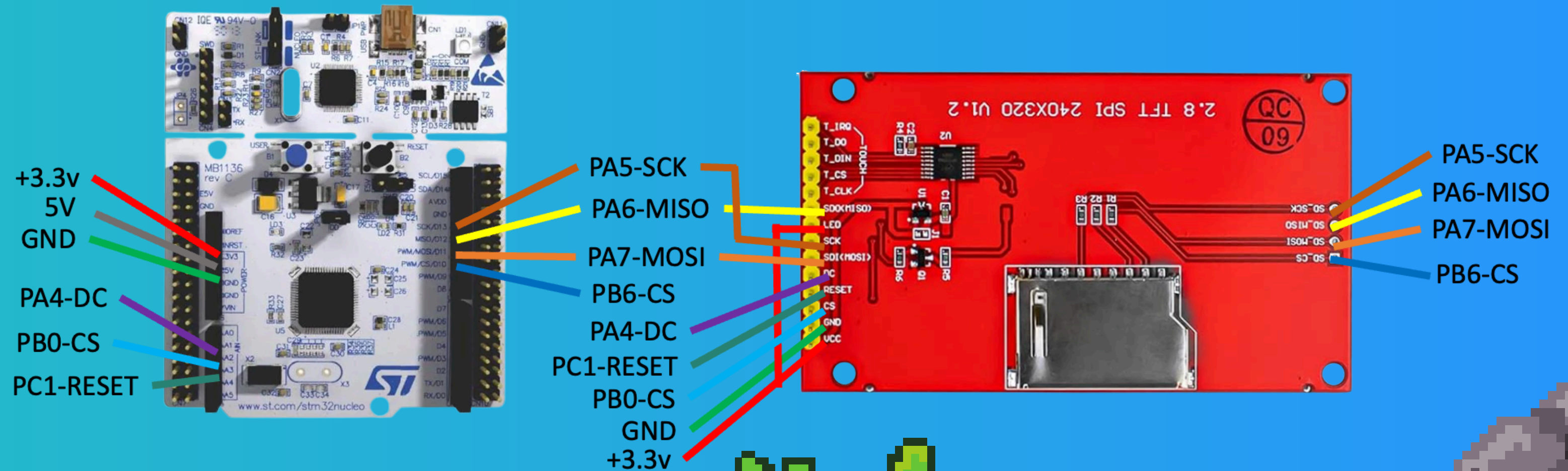
¡INTERFAZ 8 BITS

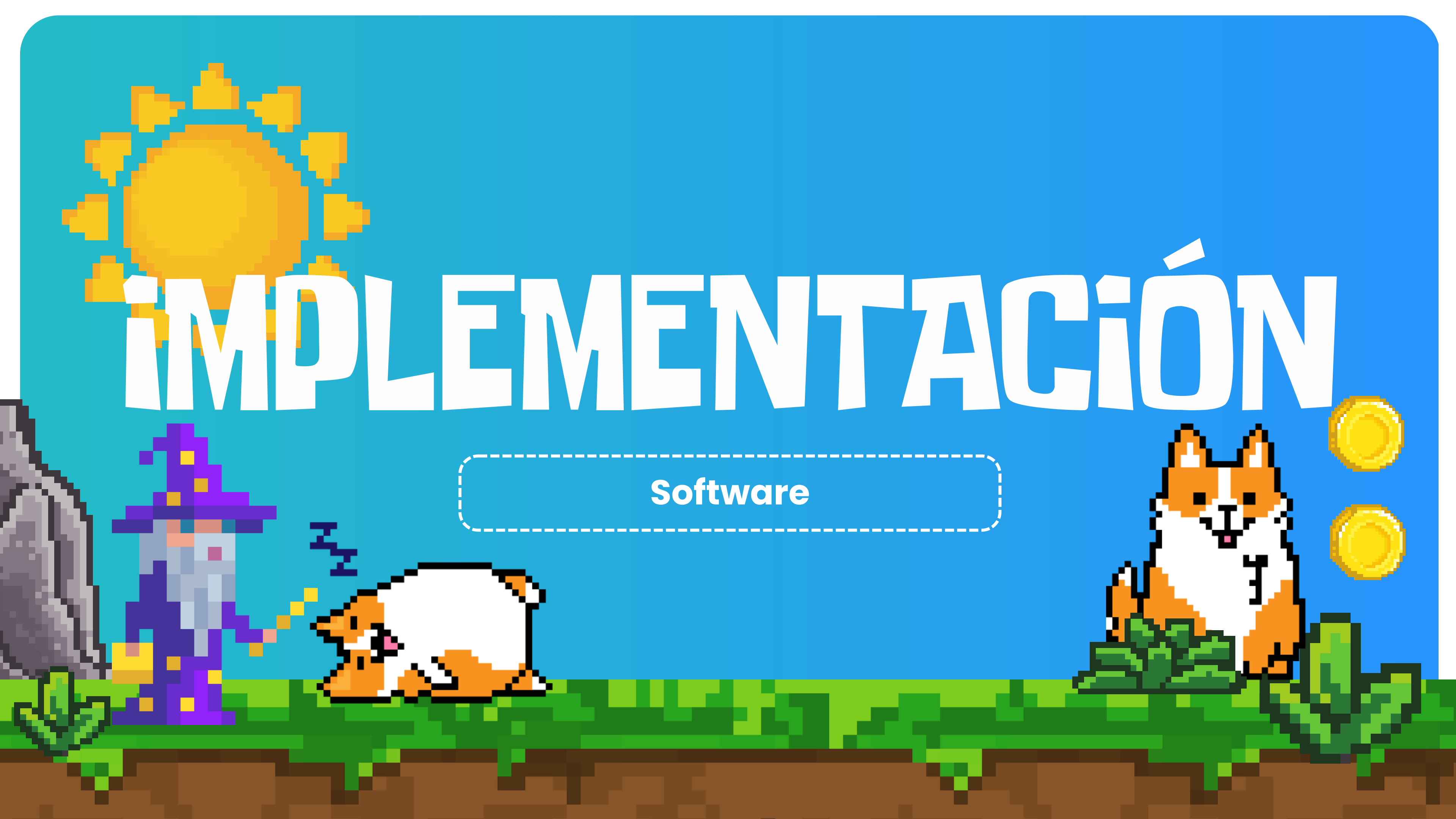
La pantalla va conectada sobre los headers tipo Arduino facilitando su conexión



¡INTERFAZ SPI

En este caso la conexión dependerá de qué módulo SPI estemos usando en el STM32F446RE, en este ejemplo se utiliza el SPI1



The background is a vibrant blue sky with a large, pixelated yellow sun in the top left. In the bottom left, a pixelated wizard in a purple robe and hat stands next to a grey rock. In the bottom center, a pixelated dog with orange and white fur is lying down. In the bottom right, a pixelated cat with orange and white fur is sitting on a green bush. Two gold coins are floating in the air near the cat. The word "IMPLEMENTACIÓN" is written in large, white, pixelated letters across the center of the image.

IMPLEMENTACIÓN

Software

FUNCIONES DE LA LIBRERÍA

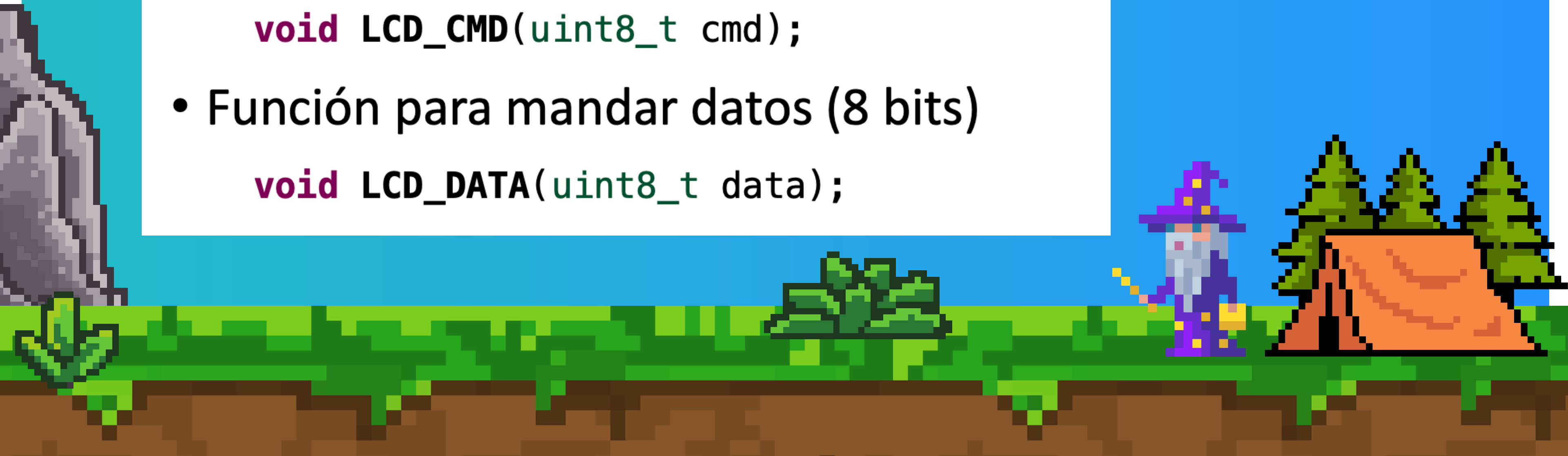
- Función para inicializar la pantalla
- Función para mandar comandos (8 bits)

```
void LCD_Init(void);
```

```
void LCD_CMD(uint8_t cmd);
```

- Función para mandar datos (8 bits)

```
void LCD_DATA(uint8_t data);
```



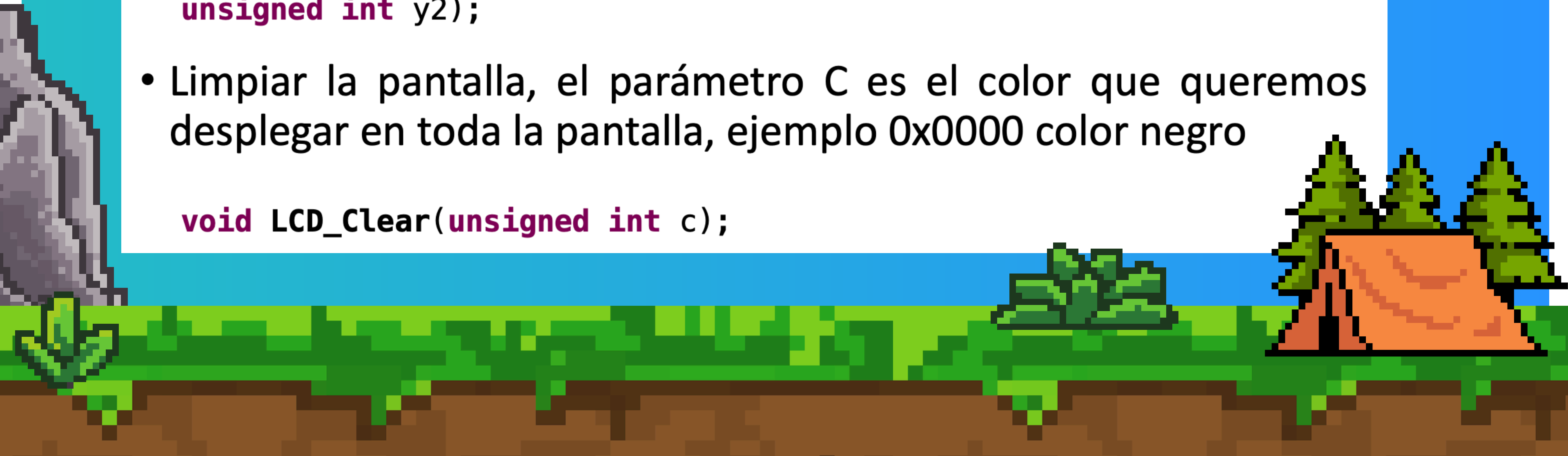
FUNCIONES DE LA LIBRERÍA

- Función para definir la Ventana en la que estaremos trabajando, donde x1, y1 son las coordenadas de inicio y x2, y2 son las coordenadas finales

```
void SetWindows(unsigned int x1, unsigned int y1, unsigned int x2,  
unsigned int y2);
```

- Limpiar la pantalla, el parámetro C es el color que queremos desplegar en toda la pantalla, ejemplo 0x0000 color negro

```
void LCD_Clear(unsigned int c);
```



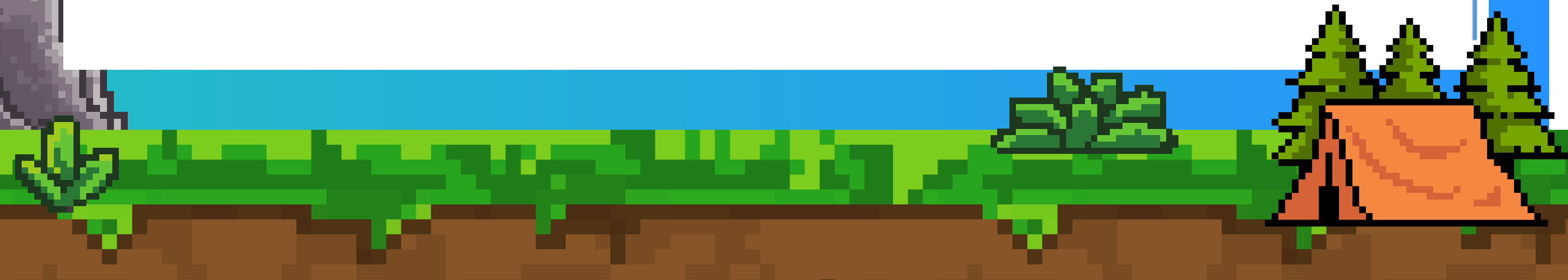
FUNCIONES DE LA LIBRERÍA

- Dibujar una línea Horizontal, x,y son las coordenadas, l la longitud y c el color de la línea

```
void H_line(unsigned int x, unsigned int y, unsigned int l, unsigned int c);
```

- Dibujar una línea Vertical, x,y son las coordenadas, l la longitud y c el color de la línea

```
void V_line(unsigned int x, unsigned int y, unsigned int l, unsigned int c);
```



FUNCIONES DE LA LIBRERÍA

- Dibujar un rectángulo, x,y son las coordenadas, w el ancho, h el alto y c el color de la línea

```
void Rect(unsigned int x, unsigned int y, unsigned int w, unsigned int h,  
unsigned int c);
```



- Dibujar un rectángulo relleno, x,y son las coordenadas, w el ancho, h el alto y c el color del relleno

```
void FillRect(unsigned int x, unsigned int y, unsigned int w, unsigned int h,  
unsigned int c);
```



FUNCIONES DE LA LIBRERÍA

- Función para colocar texto, nos pide un arreglo, las coordenadas x,y, el tamaño se tienen 3 tamaños (1 a 3), color y fondo

```
void LCD_Print(char* text, int x, int y, int fontSize, int color, int background);
```

Hola Mundo

Hola Mundo

Hola Mundo



FUNCIONES DE LA LIBRERÍA

- Función para dibujar un arreglo de colores, x, y son coordenadas, width el ancho, height el alto, y el nombre del arreglo

```
void LCD_Bitmap(unsigned int x, unsigned int y, unsigned int width,  
unsigned int height, const uint16_t *bitmap);
```



- Función para dibujar un arreglo de colores con fondo transparente, x, y son coordenadas, width el ancho, height el alto, y el nombre del arreglo, y que color es el del fondo para aplicar el transparente

```
void LCD_BitmapTransparent(uint16_t x, uint16_t y, uint16_t width,  
uint16_t height, const uint16_t *bitmap, uint16_t transparentColor);
```

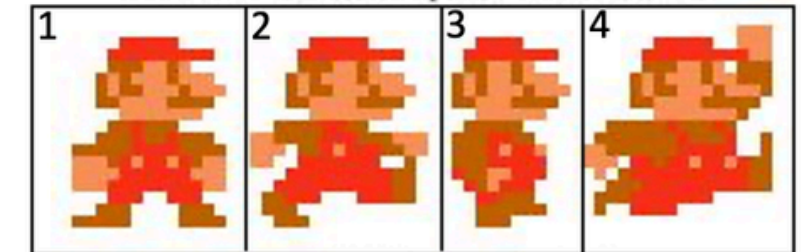


FUNCIONES DE LA LIBRERÍA

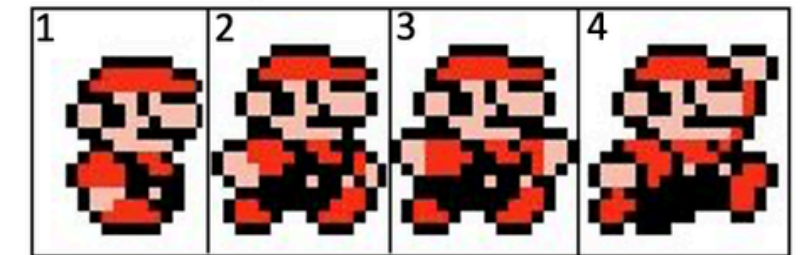
- Función para dibujar un conjunto de arreglo de colores para simular movimiento,
 - x,y son las coordenadas
 - width el ancho, height el alto
 - bitmap la variable donde se encuentra el arreglo de colores
 - Index qué columna queremos desplegar
 - Flip (0 o 1) voltear la imagen
 - Offset reservado

```
void LCD_Sprite(int x, int y, int width, int height,  
const uint16_t *bitmap, int columns, int index, char flip,  
char offset);
```

Evolución del sprite de Mario



Super Mario Bros de NES.



Super Mario Bros 3 de NES.



Super Mario World, de Super Nintendo.

BIBLIOGRAFÍA

<https://ece353.engr.wisc.edu/external-devices/ili9341/>

<https://www.stm32duino.com/viewtopic.php?t=637>

<https://gist.github.com/postmodern/ed6e670999f456ad9f13>

<https://github.com/afiskon/stm32-ili9341> [https://cdn-](https://cdn-shop.adafruit.com/datasheets/ILI9341.pdf)

[shop.adafruit.com/datasheets/ILI9341.pdf](https://cdn-shop.adafruit.com/datasheets/ILI9341.pdf)

[https://downloads.ruson.com/lcd-image-converter/lcd-image-](https://downloads.ruson.com/lcd-image-converter/lcd-image-converter-20190317.zip/preview)

[converter-20190317.zip/preview](https://downloads.ruson.com/lcd-image-converter/lcd-image-converter-20190317.zip/preview) <https://www.piskelapp.com/>

<https://www.sprites-resource.com/>

http://www.rinkydinkelectronics.com/calc_rgb565.php

